COMPTE RENDU DE TP

Yocto Project

Construire une distribution
Linux embarquée pour Raspberry Pi 3

Hamza Benjouad
Khalid Elkoussami

3e année – ISMIN
Année universitaire 2026–2027

Table des matières

Introduction **1**

1 Lab 1 – Premier build Yocto Project **3**
1.1 Mise en place de l'environnement . . . 3
1.2 Build de l'image . . . 3
1.3 Flashage de la carte SD . . . 4
1.4 Connexion série et premier boot . . . 4

2 Lab 2 – Configuration avancée, NFS et paquets **6**
2.1 Configuration réseau côté cible . . . 6
2.2 Partage de connexion et serveur NFS côté hôte . . . 6
2.3 Déploiement du *rootfs* par NFS . . . 7
2.4 Ajout de Dropbear et connexion SSH . . . 7

3 Lab 3 – Ajout d'une application personnalisée (nInvaders) **9**
3.1 Emplacement de la recette . . . 9
3.2 Écriture de la recette . . . 9
3.3 Compilation de la recette . . . 10
3.4 Intégration au *rootfs* . . . 11

4 Lab 4 – Couche personnalisée et gestion des recettes **13**
4.1 Création et enregistrement de la couche `meta-bootlinlabs` . . . 13
4.2 Déplacement de la recette `ninvaders` . . . 14
4.3 Déploiement et exécution sur la cible . . . 14

5 Lab 4+ – nano patché avec devtool **16**
5.1 Première tentative : la recette est introuvable . . . 16
5.2 Modification des sources avec devtool . . . 17
5.3 Intégration à l'image et vérification . . . 18

6 Lab 4++ – Intégration de `ctris` avec `devtool` **20**
6.1 Création de la recette avec `devtool add` . . . 20
6.2 Export de la recette avec `devtool finish` . . . 21
6.3 Ajout à l'image et reconstruction . . . 22
6.4 Déploiement du *rootfs* par NFS . . . 23
6.5 Retrouver l'adresse de la cible et lancer `ctris` . . . 23

7 Lab 5 – I^2C et init **25**
7.1 Configuration de `local.conf` . . . 25
7.2 Redéploiement du *rootfs* et vérification . . . 25
7.3 Modules du noyau non compressés . . . 26
7.4 Rattachement du fragment via un `.bbappend` . . . 26
7.5 Reconstruction de l'image . . . 26
7.6 Activation du bus I2C côté *firmware* . . . 27
7.7 Détection du capteur sur le bus . . . 27
7.8 Lecture des mesures via l'interface IIO . . . 29
7.9 Automatisation avec un script `init.d` . . . 30

7.10 Vérification après redémarrage 32

8 Lab Web – Interface web de télémétrie **33**
8.1 Architecture globale 33
8.2 Choix et configuration du serveur web 34
8.3 Script CGI de lecture du capteur 34
8.4 Page web statique 35
8.5 Résultat final 36

Conclusion **38**

Table des figures

1 Architecture globale du système . . . 1
2 Principe de fonctionnement de Yocto . . . 2

1.1 Lancement du bitbake . . . 3
1.2 Identification de la carte SD . . . 4
1.3 Flashage de la carte SD . . . 4
1.4 Connexion série . . . 4
1.5 Prompt de connexion Poky sur la cible . . . 5

2.1 Configuration réseau . . . 6
2.2 Paramètres de démarrage de la cible . . . 6
2.3 Connexion de partage PARTAGE-RPi . . . 6
2.4 Status NFS . . . 7
2.5 rootfs par NFS . . . 7
2.6 HELLO en rootfs . . . 7
2.7 Connexion SSH à la cible . . . 8

3.1 Dossier de la recette ninvaders . . . 9
3.2 Fichier ninvaders.inc . . . 10
3.3 Recette .bb . . . 10
3.4 Compilation de ninvaders réussie . . . 11
3.5 Ajout de ninvaders dans local.conf . . . 11
3.6 nInvaders sur la Raspberry Pi 3 . . . 12
3.7 Écran d'accueil de nInvaders . . . 12

4.1 Ajout d'une nouvelle couche . . . 13
4.2 Les couches dans bblayers.conf . . . 13
4.3 Vérification de la priorité de la nouvelle couche . . . 14
4.4 La couche personnalisée avec la recette nInvaders . . . 14
4.5 nInvaders exécuté sur la cible . . . 15

5.1 Recette nano introuvable, clonage de meta-openembedded . . . 17
5.2 Modification de nano avec devtool modify . . . 18
5.3 Bannière du nano patché sur la cible . . . 19

6.1 Création de la recette avec devtool add . . . 20
6.2 Contenu de workspace . . . 21
6.3 Recette générée de ctris . . . 21
6.4 Export de la recette ctris . . . 22
6.5 ctris fournie par meta-bootlinlabs . . . 22
6.6 Ajout de ctris dans local.conf . . . 22
6.7 Reconstruction de l'image . . . 23
6.8 Déploiement du rootfs dans /nfs . . . 23
6.9 Connexion SSH à la cible . . . 24
6.10 ctris en cours d'exécution sur la cible . . . 24

7.1 Ajout des configurations de I^2C . . . 25
7.2 Vérification des réglages . . . 25

7.3 Rattachement du .bbappend au noyau . . . 26
7.4 Reconstruction de l'image avec le fragment noyau . . . 27
7.5 config.txt sur la partition de boot . . . 27
7.6 Détection du capteur avec i2cdetect . . . 28
7.7 Déclaration du capteur bmp680 . . . 29
7.8 Fichiers IIO du capteur dans sysfs . . . 29
7.9 Lecture des mesures du capteur . . . 30
7.10 Script init.d bme680 . . . 31
7.11 Enregistrement du script avec update-rc.d . . . 32
7.12 Lecture de la température après redémarrage . . . 32

8.1 Architecture globale de l'interface web . . . 33
8.2 Configuration de lighttpd . . . 34
8.3 Script CGI data.cgi . . . 35
8.4 Page web index.html . . . 36
8.5 Tableau de bord BME680 Telemetry . . . 36

Introduction

Ce compte rendu décrit les manipulations réalisées lors des labs du TP Yocto Project / OpenEmbedded, sur cible Raspberry Pi 3. Pour chaque étape, on explique pourquoi la commande a été exécutée, ce qu'il a fallu configurer, ainsi que les problèmes rencontrés et leur résolution. Ce TP a été réalisé en binôme.

L'architecture matérielle du projet

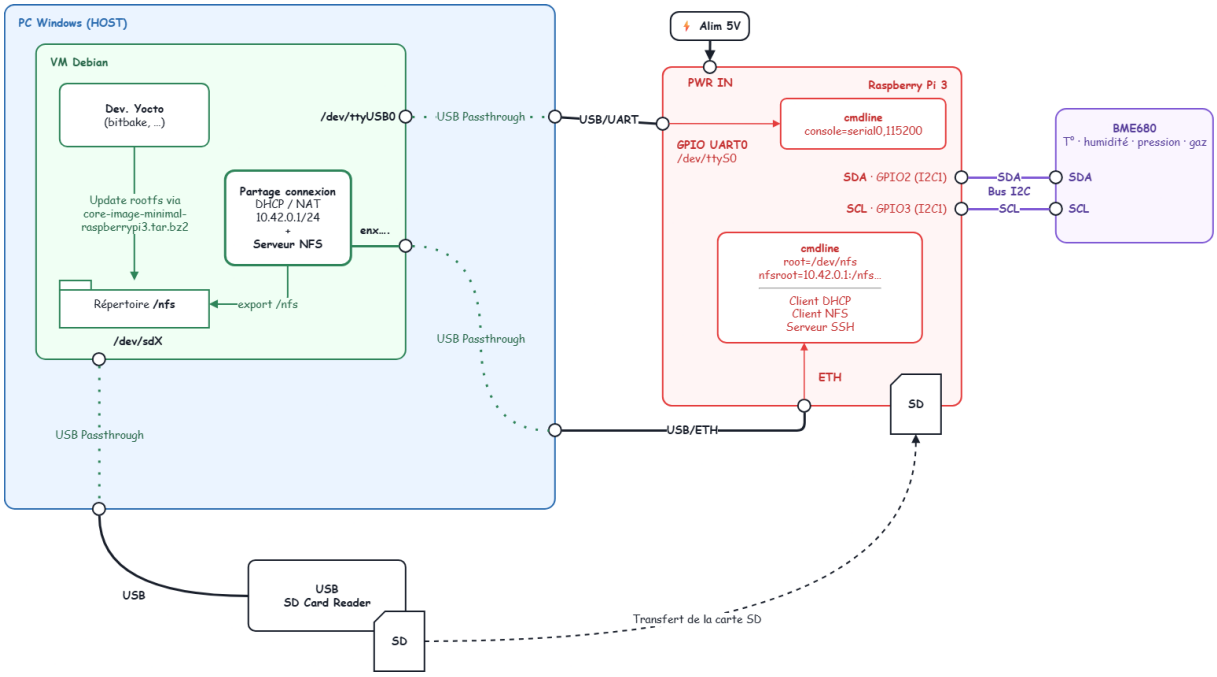


Figure 1 – Architecture globale du système

La figure présente l'architecture globale du projet. Le PC Windows héberge une machine virtuelle Debian utilisée pour le développement avec Yocto et la génération de l'image Linux destinée à la Raspberry Pi 3.

Deux liaisons assurent la communication avec la cible : une liaison USB-série pour accéder à la console et suivre le démarrage, et une liaison Ethernet pour le réseau. Cette dernière permet notamment au PC de fournir une adresse IP par DHCP et d'exporter le *rootfs* par NFS, afin que la Raspberry Pi puisse utiliser ce système de fichiers au démarrage.

La carte SD est utilisée pour le démarrage de la Raspberry Pi, tandis que le capteur BME680 est connecté à la carte par le bus I^2C via les lignes `SDA` et `SCL`. Le noyau Linux détecte ensuite le capteur et expose ses mesures à travers le sous-système IIO.

Principe général de Yocto

Avant de détailler les manipulations, voici rapidement comment Yocto fonctionne. Yocto n'est pas une distribution Linux mais un **méta-outil** qui permet de fabriquer sa propre distribution embarquée sur mesure, adaptée à une carte cible précise.

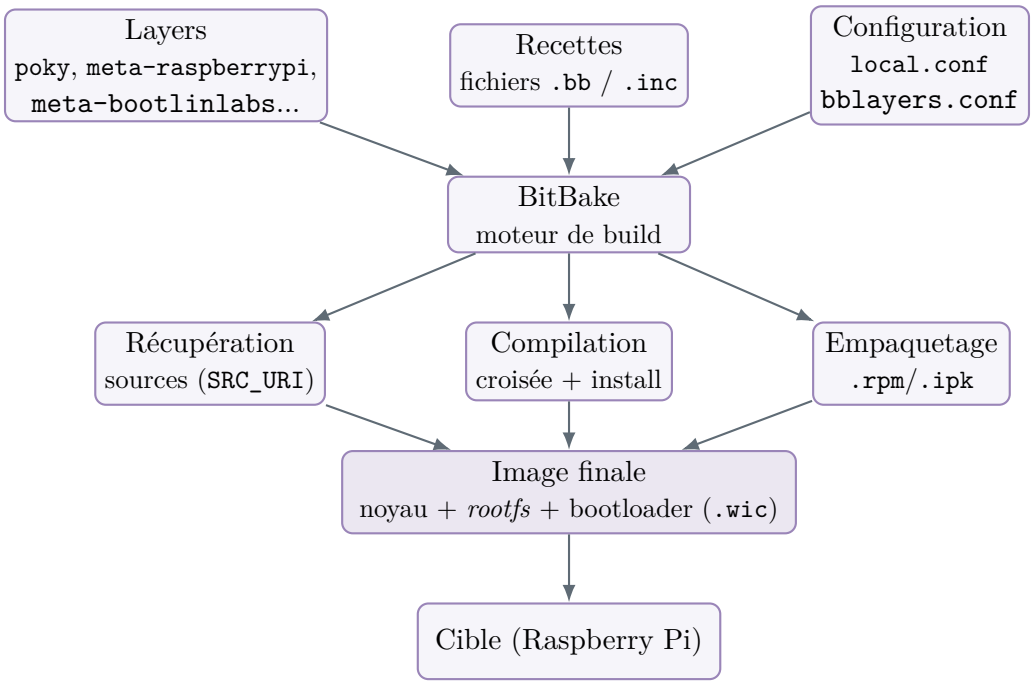


Figure 2 – Principe de fonctionnement de Yocto

En résumé : on assemble plusieurs **layers** (des ensembles de recettes, chacun apportant une brique – le support d'une carte, une application, une politique de distribution...). Chaque **recette** (`.bb`) décrit comment récupérer, compiler et installer un composant précis. Les fichiers `local.conf` et `bblayers.conf` indiquent à **BitBake** (le moteur d'orchestration) quelle machine cibler et quels layers utiliser. BitBake résout alors les dépendances entre recettes puis exécute, pour chacune, les mêmes grandes étapes : récupération des sources, compilation croisée, installation dans un répertoire temporaire, puis empaquetage. Le résultat final est une image complète (noyau, *rootfs*, bootloader) que l'on flashe ou que l'on sert par le réseau à la carte cible.

CHAPITRE 1

Lab 1 – Premier build Yocto Project

1.1 Mise en place de l'environnement

Après avoir cloné les dépôts `poky` (branche `scarthgap`) et `meta-raspberrypi`, on a initialisé l'environnement de build pour la cible `rpi3` :

```
source poky/oe-init-build-env builds/rpi3
bitbake-layers add-layer ../../meta-raspberrypi/
```

On a ajouté le layer `meta-raspberrypi` car il est indispensable : sans lui, BitBake ne connaît aucune machine nommée `raspberrypi3`, ni les recettes spécifiques au matériel (bootloader, kernel adapté, device tree).

On a ensuite complété le fichier `local.conf` pour cibler la bonne machine, et pour deux options utiles au TP :

```
MACHINE = "raspberrypi3"
INHERIT:append = " rm_work"
ENABLE_UART = "1"
```

On active `rm_work` pour supprimer le répertoire de travail de chaque paquet une fois celui-ci construit, ce qui économise beaucoup d'espace disque sur la durée du TP. On active `ENABLE_UART` pour avoir le port série de la Raspberry Pi, nécessaire pour la console de débogage utilisée juste après.

1.2 Build de l'image

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ time bitbake core-image-minimal
WARNING: Host distribution "debian-13" has not been validated with this version
of the build system; you may possibly experience unexpected failures. It is reco
mmended that you use a tested distribution.
Loading cache: 100% |                                               | ETA:  --:--:--
Loaded 0 entries from dependency cache.
Parsing recipes: 100% |#########################################| Time: 0:00:16
Parsing of 961 .bb files complete (0 cached, 961 parsed). 1923 targets, 80 skipp
ed, 0 masked, 0 errors.
NOTE: Resolving any missing task queue dependencies

Build Configuration:
BB_VERSION           = "2.8.1"
BUILD_SYS            = "x86_64-linux"
NATIVELSBSTRING      = "universal"
TARGET_SYS           = "arm-poky-linux-gnueabi"
MACHINE              = "raspberrypi3"
DISTRO               = "poky"
DISTRO_VERSION       = "5.0.20"
TUNE_FEATURES        = "arm vfp cortexa7 neon vfpv4 thumb callconvention-hard"
TARGET_FPU           = "hard"
meta
```

Figure 1.1 – Lancement du bitbake

Le `bitbake` nous permet de construire l'ensemble des paquets nécessaires (bootloader, noyau, *rootfs* minimal) : il génère en sortie une image `.wic.bz2` prête à être flashée sur une carte SD.

1.3 Flashage de la carte SD

Une fois la carte SD identifiée avec `lsblk` et démontée :

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ lsblk
NAME   MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda      8:0    0   80G  0 disk
├─sda1   8:1    0   78G  0 part /
├─sda2   8:2    0    1K  0 part
└─sda5   8:5    0    2G  0 part [SWAP]
sdb      8:16   1 14,5G  0 disk
├─sdb1   8:17   1  130M  0 part /media/dev/boot
└─sdb2   8:18   1   25M  0 part
sr0     11:0    1 1024M  1 rom
dev@debian13:~/Yocto-RPi/builds/rpi3$ umout /dev/sdb?
bash: umout : commande introuvable
dev@debian13:~/Yocto-RPi/builds/rpi3$ umount /dev/sdb?
umount: /dev/sdb2: non monté.
```

Figure 1.2 – Identification de la carte SD

On décompresse l'image et on l'écrit directement sur le périphérique bloc correspondant :

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ bzip2 -dc $BUILDDIR/tmp/deploy/images/rasp
berrypi3/core-image-minimal-raspberrypi3.rootfs.wic.bz2 | sudo dd of=/dev/sdb co
nv=fdatasync bs=4M
[sudo] Mot de passe de dev :
0+25724 enregistrements lus
0+25724 enregistrements écrits
168814592 octets (169 MB, 161 MiB) copiés, 33,0946 s, 5,1 MB/s
dev@debian13:~/Yocto-RPi/builds/rpi3$ sudo eject /dev/sdb^C
```

Figure 1.3 – Flashage de la carte SD

1.4 Connexion série et premier boot

Après avoir branché l'adaptateur USB-série, on a d'abord essayé de se connecter avec `picocom`, sans succès :

```
FATAL: cannot open /dev/ttyUSB0: Permission denied
```

Par défaut, seul l'utilisateur `root` et les membres du groupe `dialout` ont accès en lecture/écriture à `/dev/ttyUSB0`. On a donc ajouté notre utilisateur à ce groupe :

```
sudo adduser $USER dialout
```

```
dev@debian13:~$ ls -l /dev/ttyUSB0
crw-rw---- 1 root dialout 188, 0 23 sept. 13:31 /dev/ttyUSB0
dev@debian13:~$ cd Yocto-RPi/builds/rpi3/
dev@debian13:~/Yocto-RPi/builds/rpi3$ ls
bitbake-cookerdaemon.log  cache  conf  tmp
dev@debian13:~/Yocto-RPi/builds/rpi3$ cd ../../../
dev@debian13:~$ sudo adduser $USER dialout
dev@debian13:~$ cd Yocto-RPi/builds/rpi3/
dev@debian13:~/Yocto-RPi/builds/rpi3$ picocom -b 115200 /dev/ttyUSB0
picocom v3.1

port is        : /dev/ttyUSB0
```

Figure 1.4 – Connexion série

Après une déconnexion/reconnexion de session pour que l'appartenance au groupe soit prise en compte, on a relancé la connexion série, qui a cette fois fonctionné normalement, et on a vu le système démarrer jusqu'au prompt de connexion :

```
picocom -b 115200 /dev/ttyUSB0
```

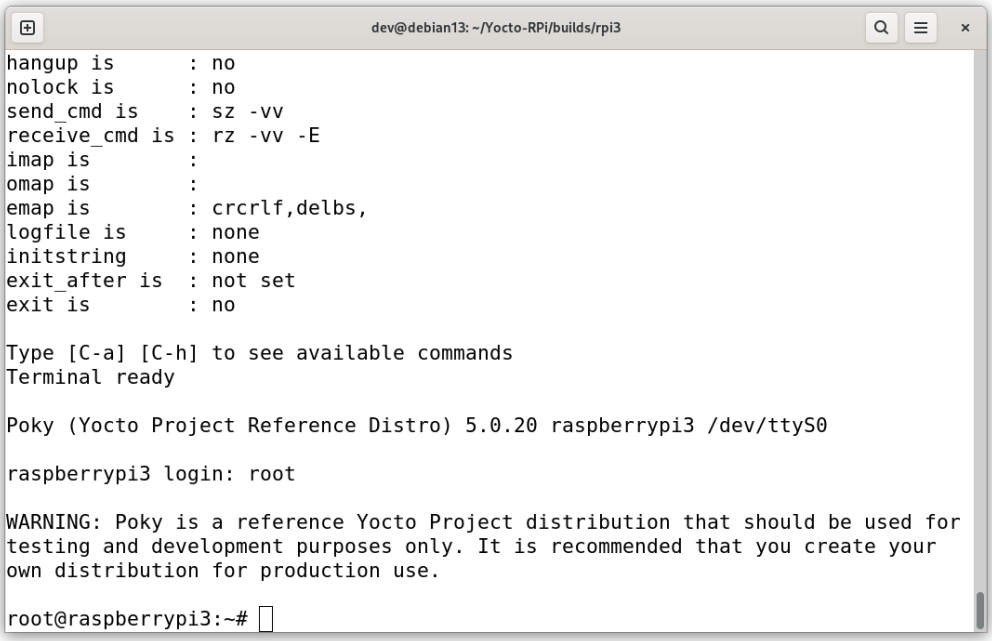


Figure 1.5 – Prompt de connexion Poky sur la cible

On retrouve bien `Poky (Yocto Project Reference Distro) 5.0.20 raspberrypi3`, et on a pu obtenir un shell root après connexion – ce qui clôt ce premier lab.

CHAPITRE 2

Lab 2 – Configuration avancée, NFS et paquets

2.1 Configuration réseau côté cible

Pour démarrer la cible avec un *rootfs* monté par le réseau (NFS) plutôt que depuis la carte SD, on a ajouté les paramètres suivants au `local.conf` (visibles dans la capture ci-dessous) :

```
MACHINE = "raspberrypi3"
INHERIT:append = " rm_work"
ENABLE_UART = "1"


CMDLINE:append = " ip=::::yoctorpi0:eth0:dhcp "
CMDLINE:append = " root=/dev/nfs nfsroot=10.42.0.1:/nfs,nfsvers=3,tcp rw"
CMDLINE:remove = "root=/dev/mmcblk0p2 rootfstype=ext4"
```

Figure 2.1 – Configuration réseau

Avec le premier `CMDLINE:append` on demande une configuration IP par DHCP sur l'interface `eth0` de la cible dès le démarrage ; avec le second on indique où trouver le *rootfs* par NFS ; avec le `CMDLINE:remove` on retire la ligne de démarrage par défaut qui pointait vers la partition de la carte SD, devenue inutile.

```
dev@debian13:~/Yocto-RPi$ nmcli device
DEVICE           TYPE      STATE                  CONNECTION
enp0s3           ethernet  connecté               Wired connection 1
lo               loopback  connecté (en externe)  lo
enx782d7e15da02  ethernet  déconnecté             --
```

Figure 2.2 – Paramètres de démarrage de la cible

2.2 Partage de connexion et serveur NFS côté hôte

Sur le PC hôte, on a configuré l'interface Ethernet reliée à la Raspberry Pi en partage de connexion (NAT + DHCP), afin de fournir une adresse IP à la cible :

```
nmcli connection add type ethernet con-name "PARTAGE-RPi" ifname enx782d7e15da02
    ↪ ipv4.method shared
```

```
dev@debian13:~/Yocto-RPi$ nmcli device
DEVICE           TYPE      STATE                  CONNECTION
enx782d7e15da02  ethernet  connecté               PARTAGE-RPi
enp0s3           ethernet  connecté               Wired connection 1
lo               loopback  connecté (en externe)  lo
```

Figure 2.3 – Connexion de partage PARTAGE-RPi

On a ensuite installé et configuré le serveur NFS pour exporter un répertoire `/nfs` :

```
sudo apt install nfs-kernel-server
sudo mkdir -m 777 /nfs
sudo exportfs -r
```

```
dev@debian13:/nfs$ sudo systemctl status nfs-kernel-server --no-pager
● nfs-server.service - NFS server and services
     Loaded: loaded (/usr/lib/systemd/system/nfs-server.service; enabled; preset
: enabled)
    Drop-In: /run/systemd/generator/nfs-server.service.d
             └─order-with-mounts.conf
     Active: active (exited) since Wed 2026-09-23 11:52:00 CEST; 2h 39min ago
 Invocation: 2f9489285ca347798556c7475be247a9
       Docs: man:rpc.nfsd(8)
             man:exportfs(8)
    Process: 1182 ExecStartPre=/usr/sbin/exportfs -r (code=exited, status=0/SUCC
ESS)
    Process: 1188 ExecStart=/bin/sh -c /usr/sbin/nfsdctl autostart || /usr/sbin/
rpc.nfsd (code=exited, status=0/SUCCESS)
   Main PID: 1188 (code=exited, status=0/SUCCESS)
   Mem peak: 1.9M
        CPU: 28ms

sept. 23 11:52:00 debian13 systemd[1]: Starting nfs-server.service - NFS se…s...
sept. 23 11:52:00 debian13 sh[1190]: nfsdctl: lockd configuration failure
```

Figure 2.4 – Status NFS

2.3 Déploiement du *rootfs* par NFS

On a décompressé le *rootfs* généré par BitBake directement dans le répertoire exporté, afin qu'il soit accessible par la cible au démarrage :

```
sudo tar xpf $BUILDDIR/tmp/deploy/images/raspberrypi3/core-image-minimal-
    ↪ raspberrypi3.rootfs.tar.bz2 -C /nfs
```

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ sudo tar xpf $BUILDDIR/tmp/deploy/images/r
aspberrypi3/core-image-minimal-raspberrypi3.rootfs.tar.bz2 -C /nfs
dev@debian13:~/Yocto-RPi/builds/rpi3$ tree /nfs/
/nfs/
├── bin
│   ├── ash -> /bin/busybox.nosuid
│   ├── base32 -> /bin/busybox.nosuid
│   ├── busybox -> busybox.nosuid
│   ├── busybox.nosuid
│   ├── busybox.suid
│   ├── cat -> /bin/busybox.nosuid
│   ├── chattr -> /bin/busybox.nosuid
│   ├── chgrp -> /bin/busybox.nosuid
│   ├── chmod -> /bin/busybox.nosuid
```

Figure 2.5 – rootfs par NFS

On a créé un fichier test (`touch /nfs/HELLO`) pour vérifier que le répertoire était bien accessible en écriture, condition nécessaire pour que la cible puisse elle aussi y écrire une fois montée en NFS.

```
dev@debian13:/nfs$ sudo touch HELLO
dev@debian13:/nfs$ ls
bin   dev  HELLO  lib    mnt   run   sys  usr
boot  etc  home   media  proc  sbin  tmp  var
```

Figure 2.6 – HELLO en rootfs

2.4 Ajout de Dropbear et connexion SSH

On a ajouté le serveur SSH léger Dropbear à la liste des paquets installés dans l'image, en complétant (et non en remplaçant) la liste par défaut dans le fichier `local.conf` :

```
IMAGE_INSTALL:append = " dropbear"
```

Après avoir reconstruit l'image et redémarré la cible sur ce nouveau *rootfs* NFS, on a testé la connexion SSH :

```
ssh root@10.42.0.159
```

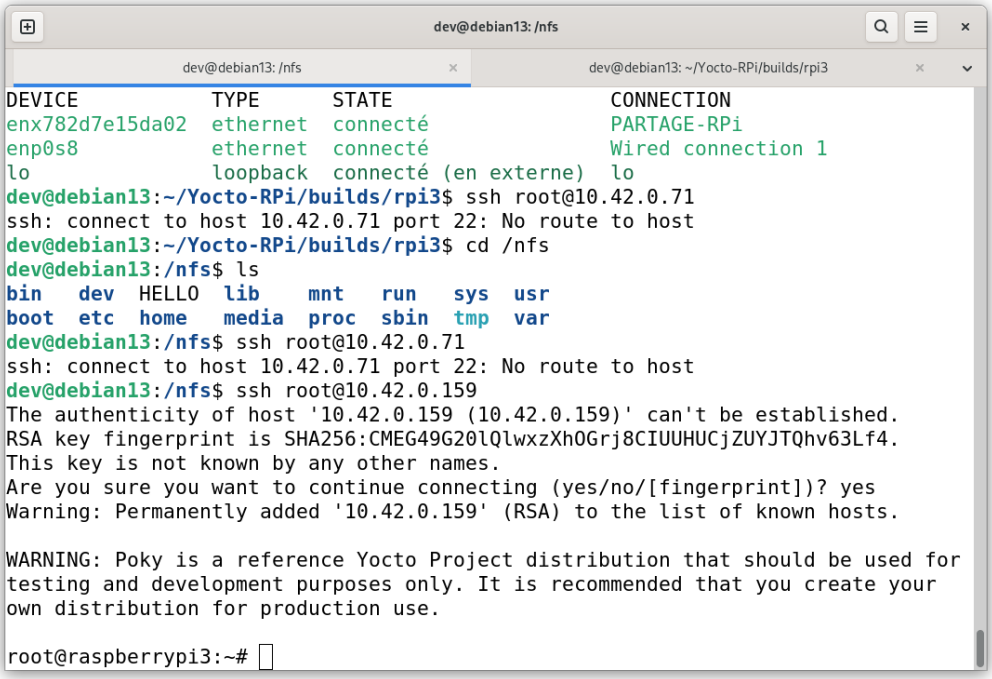


Figure 2.7 – Connexion SSH à la cible

On a d'abord essayé l'adresse `10.42.0.71`, sans succès (*No route to host*) : l'adresse avait changé entre deux attributions DHCP. La connexion a bien fonctionné une fois qu'on a utilisé la bonne adresse (`10.42.0.159`), ce qui nous a donné un shell root sur la cible via le réseau – confirmant que Dropbear tourne bien comme service.

CHAPITRE 3

Lab 3 – Ajout d'une application personnalisée (nInvaders)

3.1 Emplacement de la recette

Conformément à la convention Yocto/OpenEmbedded, on a créé un sous-dossier dédié à l'application dans `recipes-extended` :

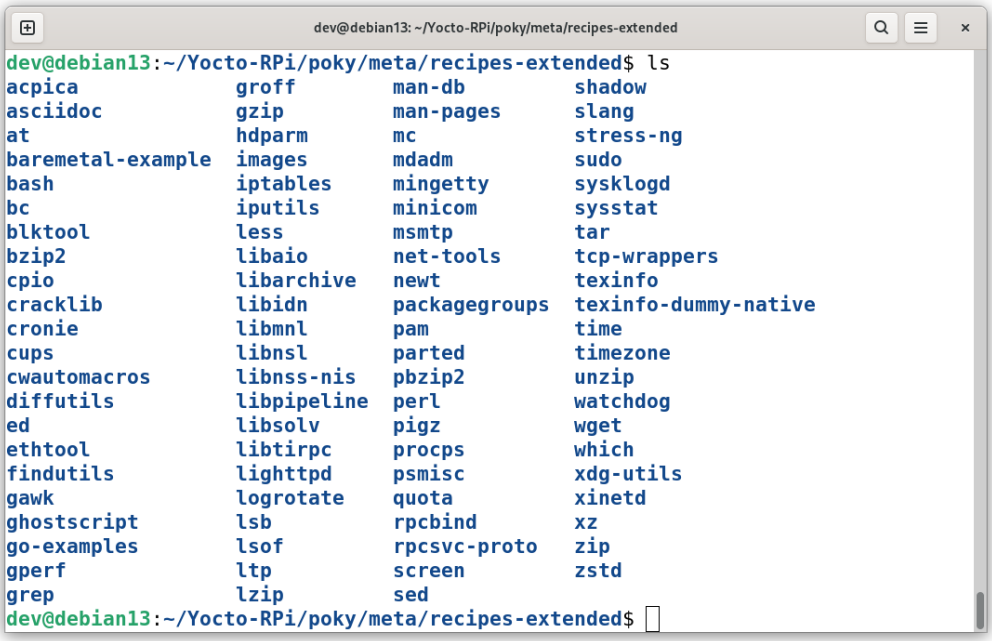


Figure 3.1 – Dossier de la recette ninvaders

```
mkdir ninvaders
cd ninvaders
```

3.2 Écriture de la recette

On a créé deux fichiers : un fichier `.inc` regroupant les métadonnées communes, et un fichier `.bb` (nommé selon la convention `$PN_$PV.bb`) qui l'inclut.

Dans `ninvaders.inc`, on définit la source du programme, sa dépendance à la bibliothèque `ncurses`, ainsi que la façon de le compiler et de l'installer :

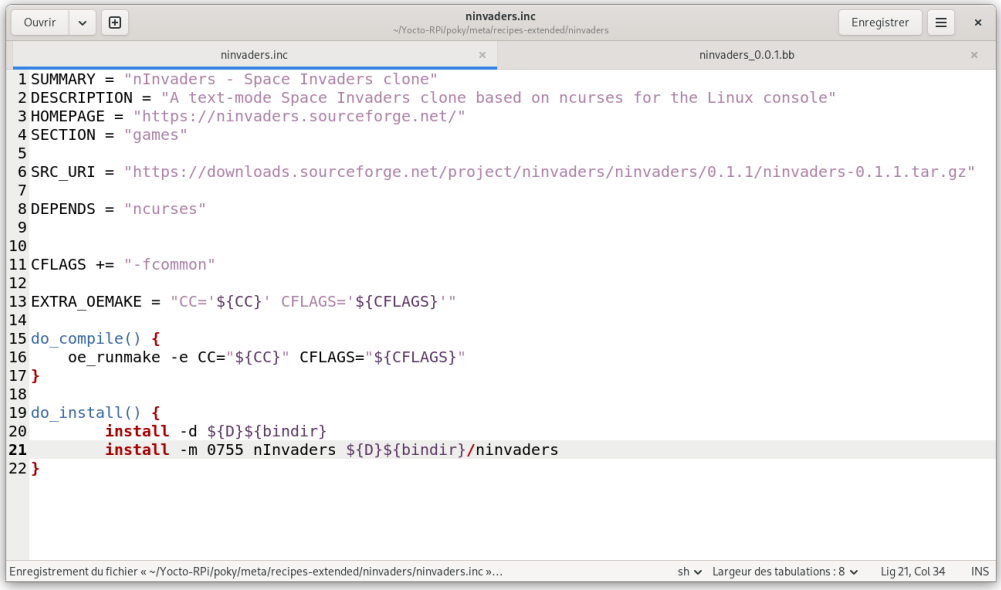


Figure 3.2 – Fichier ninvaders.inc

On a dû faire deux ajouts pour que la compilation aboutisse :

- `CFLAGS += "-fcommon"` : le code source de nInvaders étant ancien, plusieurs unités de compilation redéfinissent les mêmes symboles globaux, ce que gcc accepte de moins en moins par défaut depuis sa version 10. On restaure ainsi l'ancien comportement plus permissif.
- `EXTRA_OEMAKE = "CC='$CC' CFLAGS='$CFLAGS'"`, associé à la redéfinition de `do_compile()` avec `oe_runmake -e` : le *Makefile* original de nInvaders utilise le compilateur natif de la machine, pas le compilateur croisé fourni par Yocto. On force ainsi le *Makefile* à utiliser la bonne chaîne de compilation ARM.

Dans `ninvaders_0.0.1.bb`, on complète la recette avec la licence du programme, le *checksum* de l'archive source, et le chemin d'extraction des sources :

```
dev@debian13:~/Yocto-RPi/poky/meta/recipes-extended/ninvaders$ cat ninvaders_0.1.1.bb
require ninvaders.inc

LICENSE = "GPL-2.0-or-later"

LIC_FILES_CHKSUM = \
"file://${COMMON_LICENSE_DIR}/GPL-2.0-or-later;md5=fed54355545ffd980b814dab4a3b312c"

S = "${WORKDIR}/ninvaders-0.1.1"

SRC_URI[sha256sum] = "bfbc5c378704d9cf5e7fed288dac88859149bee5ed0850175759d310b61fd30b"
```

Figure 3.3 – Recette .bb

3.3 Compilation de la recette

```
bitbake ninvaders
```

On a dû s'y prendre en plusieurs itérations : à chaque échec, on a relancé `bitbake -c cleanall ninvaders` avant de reconstruire, afin de repartir d'un état propre. La recette compile finalement intégralement avec succès :

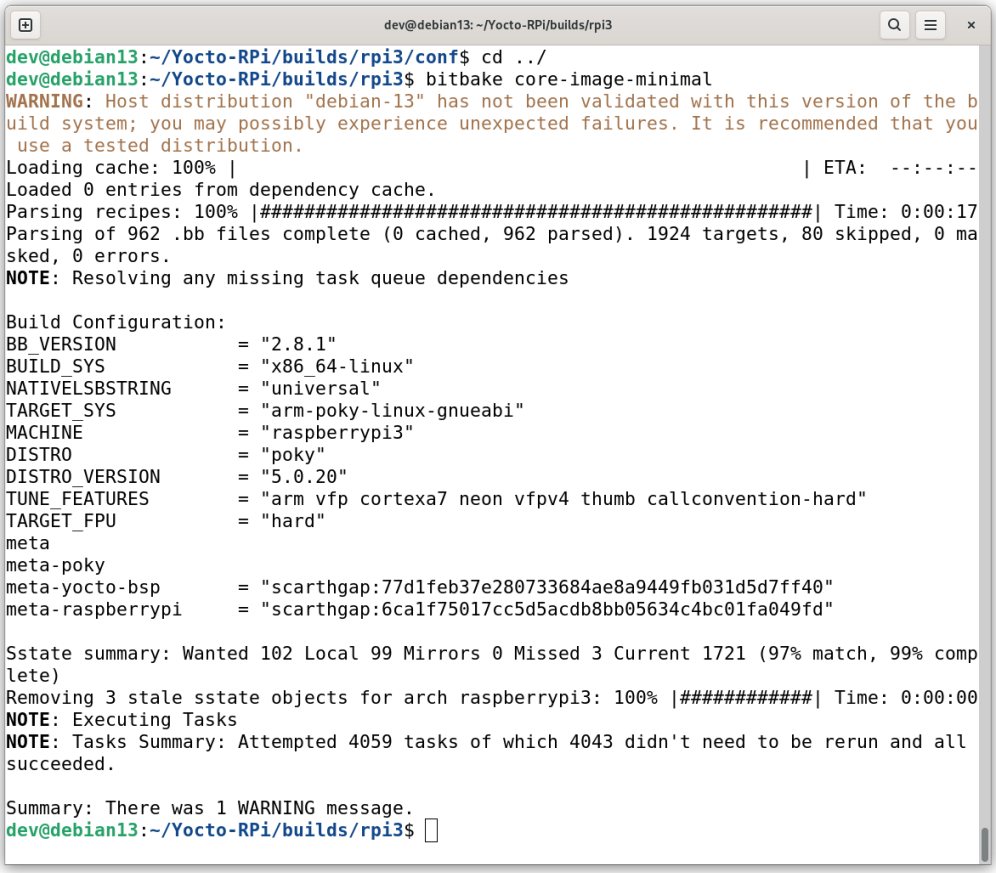


Figure 3.4 – Compilation de ninvaders réussie

3.4 Intégration au *rootfs*

On a ensuite ajouté le paquet `ninvaders` à la liste des paquets installés dans l'image, de la même façon que Dropbear au lab précédent :

```
IMAGE_INSTALL:append = " ninvaders"
```

–

```
CMDLINE:append = " ip=::::yoctorpi0:eth0:dhcp "
CMDLINE:append = " root=/dev/nfs nfsroot=10.42.0.1:/nfs,nfsvers=3,tcp rw"
CMDLINE:remove = "root=/dev/mmcblk0p2 rootfstype=ext4"

IMAGE_INSTALL:append = " dropbear"
IMAGE_INSTALL:append = " ninvaders"
dev@debian13:~/Yocto-RPi/builds/rpi3/conf$
```

Figure 3.5 – Ajout de ninvaders dans local.conf

Après avoir reconstruit l'image (`bitbake core-image-minimal`) et mis à jour le *rootfs* exporté par NFS, on a vérifié la présence du binaire, puis lancé le programme via une connexion SSH sur la cible :

```
find /nfs -name ninvaders
ssh root@10.42.0.159
which ninvaders
ninvaders
```

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ ssh root@10.42.0.159

WARNING: Poky is a reference Yocto Project distribution that should be used for
testing and development purposes only. It is recommended that you create your
own distribution for production use.

root@raspberrypi3:~# which ninvaders
/usr/bin/ninvaders
root@raspberrypi3:~# ninvaders
root@raspberrypi3:~#
```

Figure 3.6 – nInvaders sur la Raspberry Pi 3

Le jeu se lance correctement sur la cible, comme le montre son écran d'accueil qu'on a pu afficher sur la console SSH :

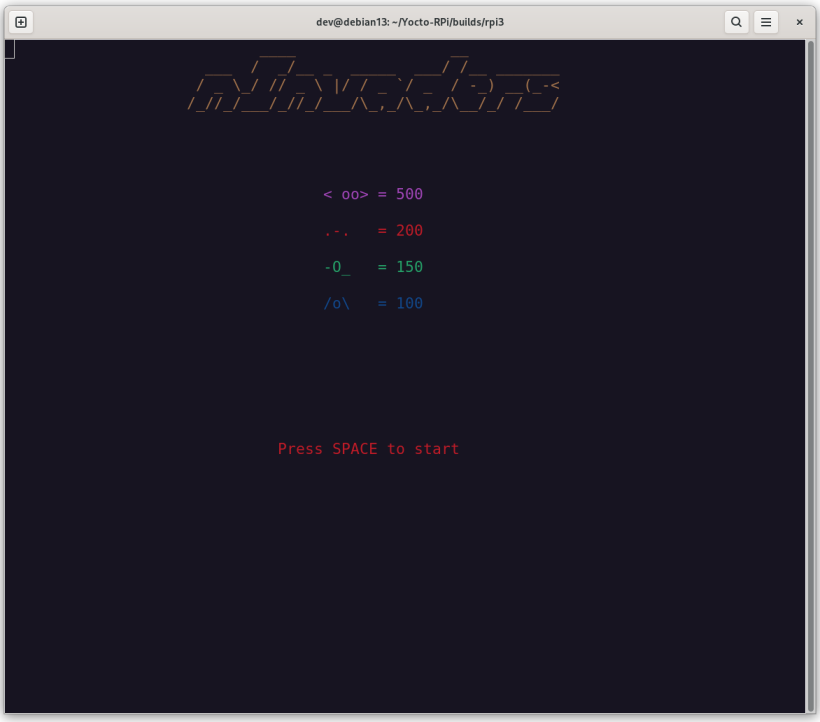


Figure 3.7 – Écran d'accueil de nInvaders

Cette dernière capture confirme que l'application, cross-compilée par Yocto pour l'architecture ARM de la Raspberry Pi 3, est bien fonctionnelle sur la cible réelle.

CHAPITRE 4

Lab 4 – Couche personnalisée et gestion des recettes

Afin de ne pas polluer l'arborescence de Poky (`meta`), les recettes personnalisées ainsi que nos applications métier doivent être isolées dans une couche dédiée. Dans ce lab, nous avons créé la couche `meta-bootlinlabs` et y avons déplacé la recette `ninvaders`. L'intégration du jeu `ctris` avec `devtool` fait l'objet du lab 4++.

4.1 Création et enregistrement de la couche `meta-bootlinlabs`

Nous avons généré la structure de la couche avec une priorité fixée à 7 (supérieure à Poky à 5, mais inférieure à `meta-raspberrypi` à 9), puis nous l'avons ajoutée à la configuration :

```
cd ~/Yocto-RPi
bitbake-layers create-layer --priority 7 meta-bootlinlabs
bitbake-layers add-layer ~/Yocto-RPi/meta-bootlinlabs
bitbake-layers show-layers
```

```
dev@debian13:~/Yocto-RPi$ cd builds/rpi3/
dev@debian13:~/Yocto-RPi/builds/rpi3$ bitbake-layers add-layer /home/dev
/Yocto-RPi/meta-bootlinlabs
NOTE: Starting bitbake server...
dev@debian13:~/Yocto-RPi/builds/rpi3$ cd ../../
dev@debian13:~/Yocto-RPi$ bitbake-layers show-layers
NOTE: Starting bitbake server...
layer                 path
                      priority
==========================================================================
================================
core                  /home/dev/Yocto-RPi/poky/meta
                      5
yocto                 /home/dev/Yocto-RPi/poky/meta-poky
                      5
yoctobsp              /home/dev/Yocto-RPi/poky/meta-yocto-bsp
                      5
raspberrypi           /home/dev/Yocto-RPi/meta-raspberrypi
                      9
meta-bootlinlabs      /home/dev/Yocto-RPi/meta-bootlinlabs
                      7
```

Figure 4.1 – Ajout d'une nouvelle couche

```
dev@debian13:~/Yocto-RPi/builds/rpi3/conf$ cat bblayers.conf
# POKY_BBLAYERS_CONF_VERSION is increased each time build/conf/bblayers.conf
# changes incompatibly
POKY_BBLAYERS_CONF_VERSION = "2"

BBPATH = "${TOPDIR}"
BBFILES ?= ""

BBLAYERS ?= " \
  /home/dev/Yocto-RPi/poky/meta \
  /home/dev/Yocto-RPi/poky/meta-poky \
  /home/dev/Yocto-RPi/poky/meta-yocto-bsp \
  /home/dev/Yocto-RPi/meta-raspberrypi \
  /home/dev/Yocto-RPi/builds/rpi3/workspace \
  /home/dev/Yocto-RPi/meta-bootlinlabs \
  "
```

Figure 4.2 – Les couches dans bblayers.conf

Le fichier `conf/layer.conf` généré configure la recherche des recettes et la compatibilité :

```
dev@debian13:~/Yocto-RPi/meta-bootlinlabs/conf$ cat layer.conf
# We have a conf and classes directory, add to BBPATH
BBPATH .= ":${LAYERDIR}"

# We have recipes-* directories, add to BBFILES
BBFILES += "${LAYERDIR}/recipes-*/*/*.bb \
            ${LAYERDIR}/recipes-*/*/*.bbappend"

BBFILE_COLLECTIONS += "meta-bootlinlabs"
BBFILE_PATTERN_meta-bootlinlabs = "^${LAYERDIR}/"
BBFILE_PRIORITY_meta-bootlinlabs = "7"

LAYERDEPENDS_meta-bootlinlabs = "core"
LAYERSERIES_COMPAT_meta-bootlinlabs = "scarthgap"
```

Figure 4.3 – Vérification de la priorité de la nouvelle couche

4.2 Déplacement de la recette `ninvaders`

La recette de `ninvaders` précédemment créée dans `poky/meta` a été déplacée dans notre nouvelle couche :

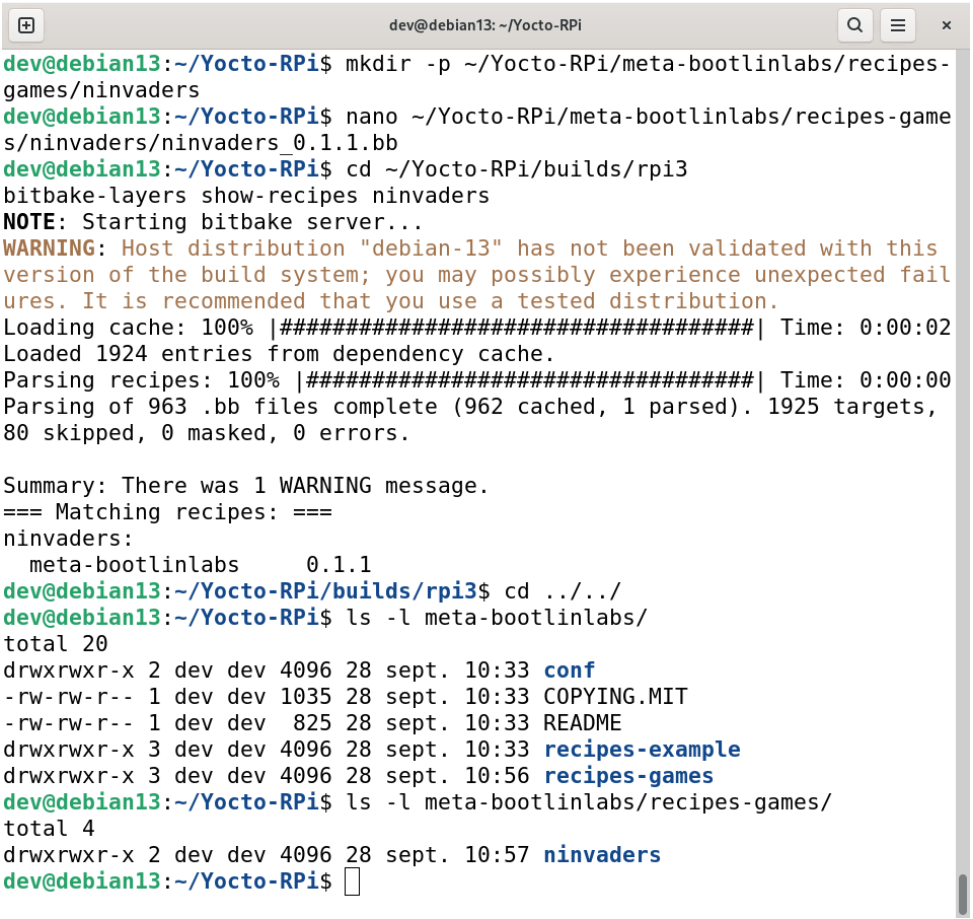


Figure 4.4 – La couche personnalisée avec la recette nInvaders

4.3 Déploiement et exécution sur la cible

L'application a été ajoutée à l'image via le fichier `conf/local.conf` :

```
IMAGE_INSTALL:append = " ninvaders"
```

Après compilation de l'image et mise à jour du répertoire racine NFS, le jeu est fonctionnel sur la carte Raspberry Pi 3.

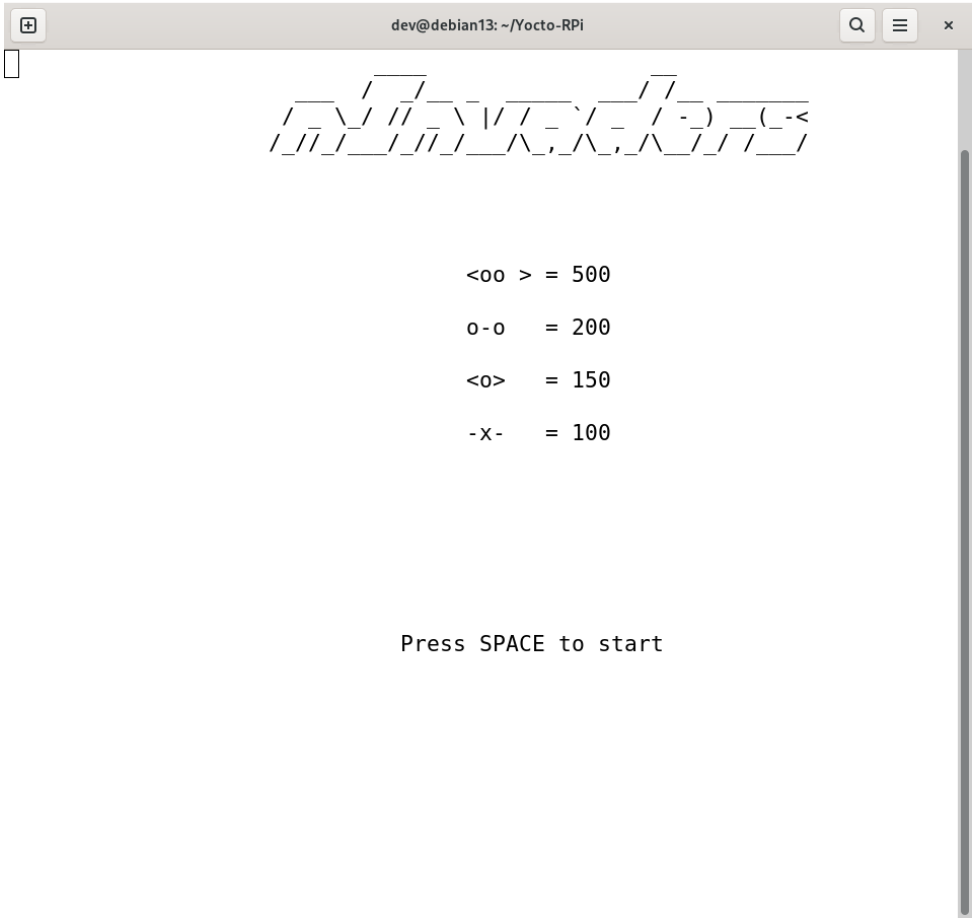


Figure 4.5 – nInvaders exécuté sur la cible

CHAPITRE 5

Lab 4+ – nano patché avec devtool

Ce premier bonus consiste à ajouter l'éditeur `nano` à l'image, mais dans une version *patchée* : on modifie son message d'accueil, pour vérifier que l'on sait intervenir sur le code d'un logiciel tiers à travers Yocto.

5.1 Première tentative : la recette est introuvable

On a voulu préparer la modification de `nano` avec `devtool modify nano`. La commande échoue :

```
ERROR: Unable to find any recipe file matching "nano"
```

Aucune des couches actives ne fournit de recette `nano` : elle n'est pas dans Poky (`meta`), mais dans `meta-openembedded`. On l'a confirmé en constatant que le dossier `poky/meta-openembedded/meta-oe` n'existe pas. On a donc cloné le dépôt, sur la même branche que Poky (*scarthgap*) pour rester compatible, puis ajouté la seule couche dont on a besoin, `meta-oe` (le dépôt en contient plusieurs) :

```
cd ~/Yocto-RPi/poky
git clone -b scarthgap https://git.openembedded.org/meta-openembedded
cd ~/Yocto-RPi/builds/rpi3
bitbake-layers add-layer ~/Yocto-RPi/poky/meta-openembedded/meta-oe
```

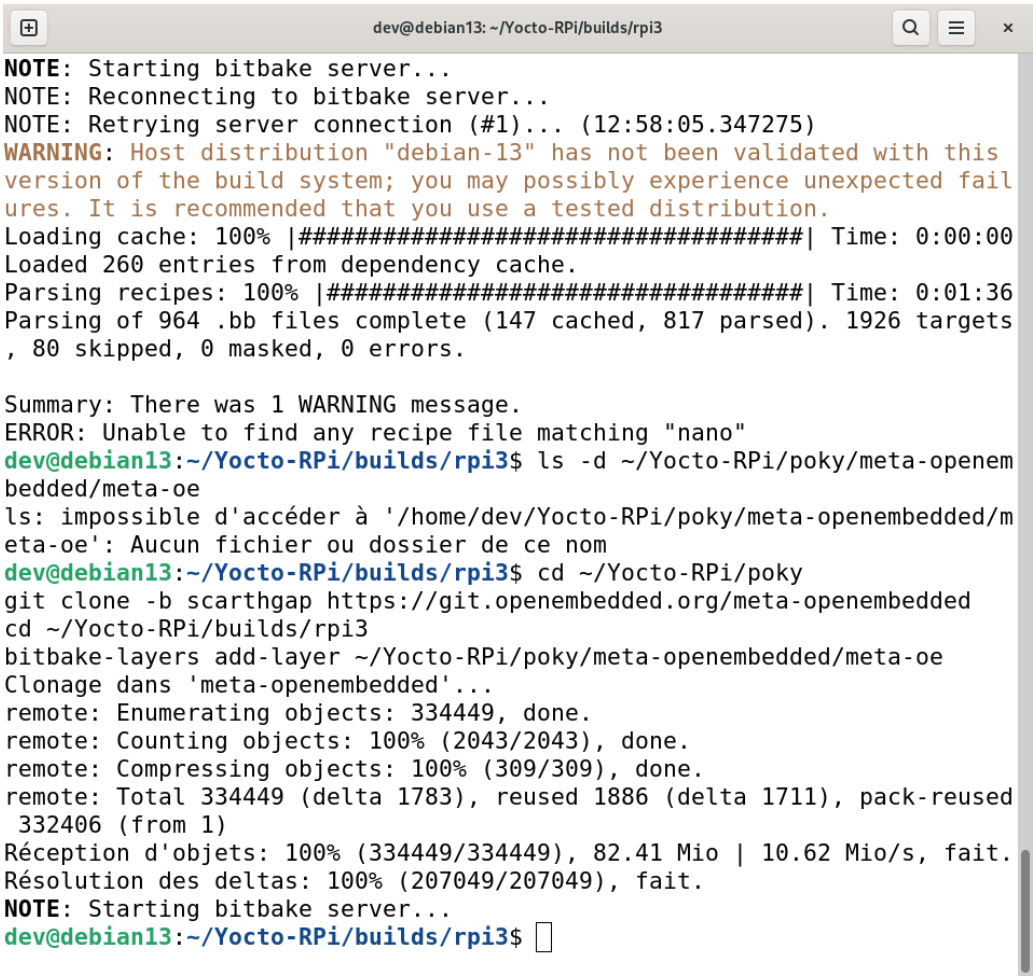


Figure 5.1 – Recette nano introuvable, clonage de meta-openembedded

5.2 Modification des sources avec devtool

Une fois `meta-oe` ajoutée, `devtool modify nano` a pu s'exécuter. Il extrait les sources de `nano` dans `builds/rpi3/workspace/sources/nano` et crée une couche temporaire, `workspace`, dont la recette de `nano` pointe désormais vers ce dossier. On la voit apparaître dans le bloc *Build Configuration*, à côté de `meta-oe` (`scarthgap:b5874ea...`).

L'intérêt de `devtool` est de travailler directement sur des sources modifiables, sans écrire un patch à la main : on édite le code dans le *workspace* puis on reconstruit, et c'est cette version modifiée qui est compilée. On y a changé le message affiché au démarrage de l'éditeur.

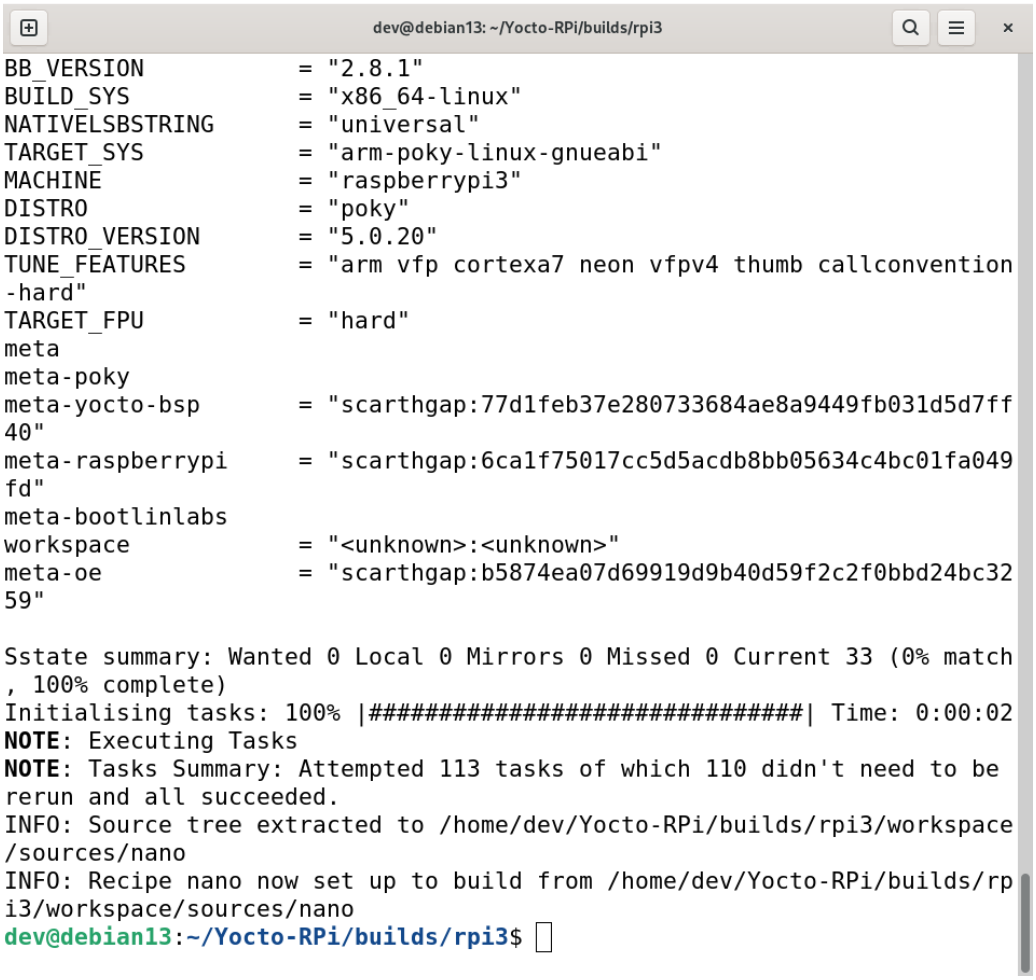


Figure 5.2 – Modification de nano avec devtool modify

5.3 Intégration à l'image et vérification

On a ajouté `nano` à la liste des paquets de l'image, de la même façon que `dropbear` et `ninvaders` (la ligne devient `IMAGE_INSTALL:append = " dropbear nano i2c-tools"`). Après reconstruction de l'image, redéploiement du *rootfs* dans `/nfs` et redémarrage de la cible, on obtient le prompt de connexion.

Le démarrage affiche au passage quelques messages réseau (`udhcpc: no lease`, puis `ip: SIOCGIFFLAGS: No such device`), mais ils n'ont pas empêché le système de démarrer jusqu'au login. Une fois connecté en root, on vérifie la présence du binaire :

```
which nano
/usr/bin/nano
```

En lançant `nano`, la bannière annonce bien notre modification : *Welcome to my patched version of nano*. On remarque aussi la version, GNU nano 7.2 : c'est celle de la recette *scarthgap*, alors que l'éditeur de l'hôte Debian est en 8.4 (visible sur la capture de `local.conf` plus bas). Il s'agit donc bien du binaire compilé par Yocto pour la cible.

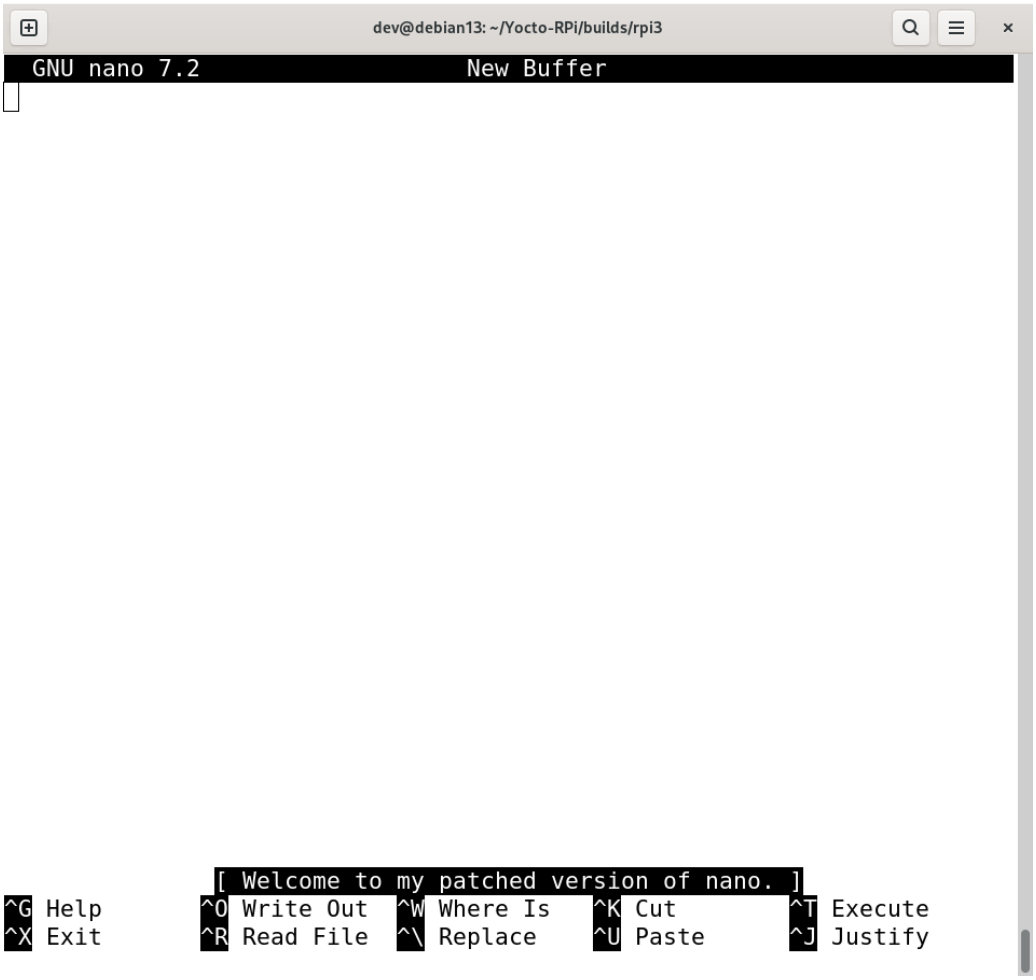


Figure 5.3 – Bannière du nano patché sur la cible

CHAPITRE 6

Lab 4++ – Intégration de `ctris` avec `devtool`

Dans ce lab, on ajoute le jeu `ctris` (un clone de Tetris pour terminal) à l'image, mais cette fois sans écrire la recette entièrement à la main : on utilise `devtool`, qui génère une recette de départ à partir du dépôt distant, permet de travailler dessus dans un espace de travail temporaire, puis la range dans notre couche.

6.1 Création de la recette avec `devtool add`

Depuis le répertoire de build, on demande à `devtool` de créer une recette `ctris` à partir du dépôt GitHub du projet :

```
cd ~/Yocto-RPi/builds/rpi3
devtool add ctris https://github.com/0xminik/ctris.git
```

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ devtool add ctris https://github.com/0xminik/ctris
.git
NOTE: Starting bitbake server...
INFO: Creating workspace layer in /home/dev/Yocto-RPi/builds/rpi3/workspace
NOTE: Reconnecting to bitbake server...
NOTE: Retrying server connection (#1)... (13:27:58.991962)
NOTE: Reconnecting to bitbake server...
NOTE: Reconnecting to bitbake server...
NOTE: Retrying server connection (#1)... (13:27:58.991962)
NOTE: Retrying server connection (#1)... (13:27:58.991962)
NOTE: Starting bitbake server...
NOTE: Reconnecting to bitbake server...
NOTE: Retrying server connection (#1)... (13:27:59.624820)
NOTE: Reconnecting to bitbake server...
NOTE: Reconnecting to bitbake server...
NOTE: Retrying server connection (#1)... (13:27:59.624820)
NOTE: Retrying server connection (#1)... (13:27:59.624820)
NOTE: Starting bitbake server...
INFO: Fetching git://github.com/0xminik/ctris.git;protocol=https;branch=master...
WARNING: Host distribution "debian-13" has not been validated with this version of the b
uild system; you may possibly experience unexpected failures. It is recommended that you
 use a tested distribution.
Loading cache: 100% |                                                | ETA:  --:--:--
Loaded 0 entries from dependency cache.
Parsing recipes:  24% |############                                    | ETA:  0:00:19
```

Figure 6.1 – Création de la recette avec devtool add

Cette commande a plusieurs effets, visibles dans la sortie :

- Elle crée une couche temporaire `workspace` dans `builds/rpi3/workspace`.
- Elle récupère les sources depuis le dépôt Git.
- Elle analyse ensuite toutes les recettes (*Parsing recipes*). Le cache de dépendances est vide à ce moment, donc l'analyse est complète et prend un peu de temps.

On a ensuite vérifié le contenu du *workspace* :

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ ls workspace/
appends/      conf/         .devtool_md5  README        recipes/      sources/
dev@debian13:~/Yocto-RPi/builds/rpi3$ cd ../../
dev@debian13:~/Yocto-RPi$ ls
build  builds  downloads  meta-bootlinlabs  meta-raspberrypi  poky  rpi3  sstate-cache
dev@debian13:~/Yocto-RPi$ ls meta-bootlinlabs/
conf/            COPYING.MIT      README           recipes-example/
```

Figure 6.2 – Contenu de workspace

La recette générée se trouve dans le *workspace* (`workspace/recipes/ctris/ctris_git.bb`). Elle est incomplète : on l'a donc complétée en ajoutant la dépendance à `ncurses` et en passant `CFLAGS` et `LDFLAGS` au compilateur, pour respecter les contrôles qualité de Yocto :

```
dev@debian13:~/Yocto-RPi/meta-bootlinlabs/recipes-games/ctris$ cat ctris_git.bb
# Recipe created by recipetool
# This is the basis of a recipe and may need further editing in order to be fully functional.
# (Feel free to remove these comments when editing.)

# WARNING: the following LICENSE and LIC_FILES_CHKSUM values are best guesses - it is
# your responsibility to verify that the values are complete and correct.
SUMMARY = "Color Tetris clone for the terminal"
HOMEPAGE = "https://github.com/0xminik/ctris"
LICENSE = "GPL-2.0-only"
LIC_FILES_CHKSUM = "file://COPYING;md5=0636e73ff0215e8d672dc4c32c317bb3 \
                    file://LICENSE;md5=2c1c00f9d3ed9e24fa69b932b7e7aff2"

SRC_URI = "git://github.com/0xminik/ctris.git;protocol=https;branch=master"

# Modify these as desired
PV = "1.0+git"
SRCREV = "9db20c4aa60589bc9ea1509d9c81519b1ccde526"

S = "${WORKDIR}/git"

# NOTE: this is a Makefile-only piece of software, so we cannot generate much of the
# recipe automatically - you will need to examine the Makefile yourself and ensure
# that the appropriate arguments are passed in.

DEPENDS = "ncurses"


do_compile () {
    oe_runmake CC="${CC}" CFLAGS="${CFLAGS}" LDFLAGS="${LDFLAGS}" LIBS="-lncurses ${LDFLAGS}"
}

do_install () {
    install -d ${D}${bindir}
    install -m 0755 ctris ${D}${bindir}/
}
```

Figure 6.3 – Recette générée de ctris

6.2 Export de la recette avec `devtool finish`

Une fois la recette au point, on l'exporte vers notre couche avec :

```
devtool finish ctris /home/dev/Yocto-RPi/meta-bootlinlabs/recipes-games/ctris/
```

```
dev@debian13:~/Yocto-RPi$ devtool finish ctris /home/dev/Yocto-RPi/meta-bootlinlabs/reci
pes-games/ctris/
NOTE: Starting bitbake server...
NOTE: Reconnecting to bitbake server...
NOTE: Retrying server connection (#1)... (13:51:29.848332)
NOTE: Reconnecting to bitbake server...
NOTE: Previous bitbake instance shutting down?, waiting to retry... (13:51:29.849090)
NOTE: Directory listing: ['cache', 'bitbake-cookerdaemon.log', 'tmp', 'conf', 'workspace
']
NOTE: Retrying server connection (#2)... (13:51:29.986248)
WARNING: Host distribution "debian-13" has not been validated with this version of the b
uild system; you may possibly experience unexpected failures. It is recommended that you
 use a tested distribution.
Loading cache: 100% |################################################| Time: 0:00:00
Loaded 1925 entries from dependency cache.
Parsing recipes: 100% |##############################################| Time: 0:00:00
Parsing of 964 .bb files complete (963 cached, 1 parsed). 1926 targets, 80 skipped, 0 ma
sked, 0 errors.

Summary: There was 1 WARNING message.
INFO: Updating SRCREV in recipe ctris_git.bb
INFO: Moving recipe file to /home/dev/Yocto-RPi/meta-bootlinlabs/recipes-games/ctris
INFO: Preserving source tree in /home/dev/Yocto-RPi/builds/rpi3/workspace/attic/sources/
ctris.20260929135132
If you no longer need it then please delete it manually.
It is also possible to reuse it via devtool source tree argument.
```

Figure 6.4 – Export de la recette ctris

On vérifie que la recette existe bien dans notre couche :

```
bitbake-layers show-recipes ctris
```

```
dev@debian13:~/Yocto-RPi$ bitbake-layers show-recipes ctris
NOTE: Starting bitbake server...
WARNING: Host distribution "debian-13" has not been validated with this version of the b
uild system; you may possibly experience unexpected failures. It is recommended that you
 use a tested distribution.
Loading cache: 100% |################################################| Time: 0:00:00
Loaded 1925 entries from dependency cache.
Parsing recipes: 100% |##############################################| Time: 0:00:00
Parsing of 964 .bb files complete (963 cached, 1 parsed). 1926 targets, 80 skipped, 0 ma
sked, 0 errors.

Summary: There was 1 WARNING message.
=== Matching recipes: ===
ctris:
  meta-bootlinlabs     1.0+git
```

Figure 6.5 – ctris fournie par meta-bootlinlabs

La sortie indique `ctris` en version `1.0+git`, fournie par `meta-bootlinlabs`.

6.3 Ajout à l'image et reconstruction

On ajoute le paquet à l'image dans `conf/local.conf`, comme pour `dropbear` et `ninvaders` :

```
IMAGE_INSTALL:append = " ctris"
```

```
IMAGE_INSTALL:append = " dropbear"
IMAGE_INSTALL:append = " ninvaders"
RM_WORK_EXCLUDE += "ninvaders"
IMAGE_INSTALL:append = " ctris"
```

Figure 6.6 – Ajout de ctris dans local.conf

On reconstruit ensuite l'image :

```
bitbake core-image-minimal
```

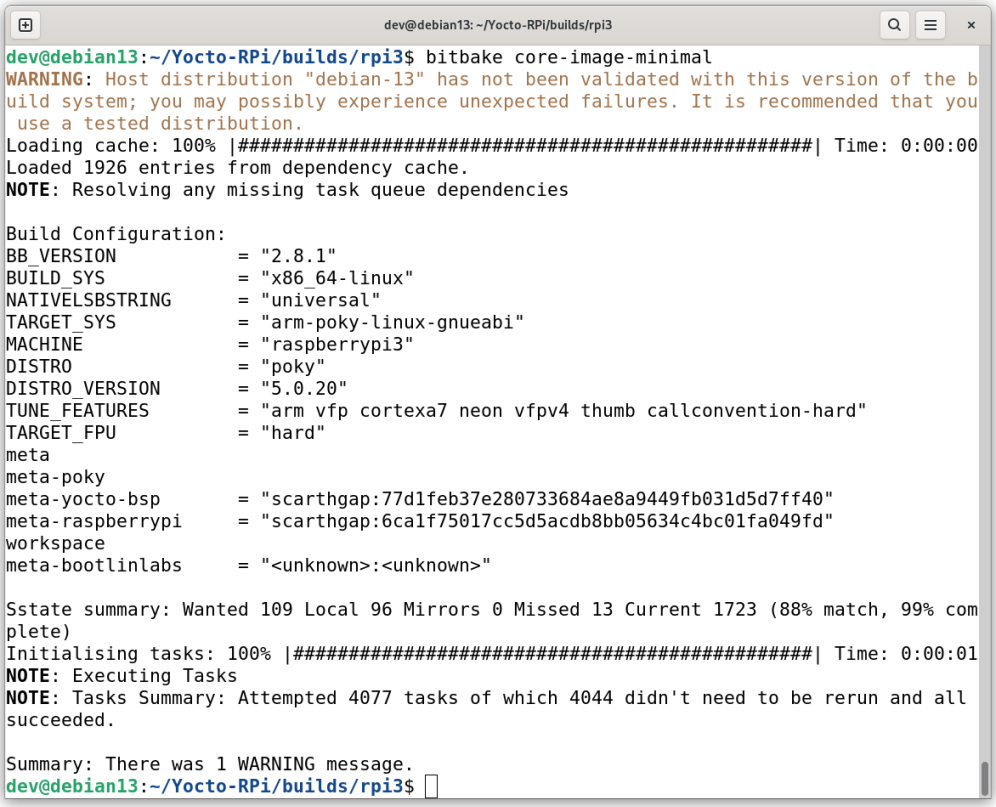


Figure 6.7 – Reconstruction de l'image

6.4 Déploiement du *rootfs* par NFS

On extrait le nouveau *rootfs* dans le répertoire exporté `/nfs` :

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ sudo tar xpf $BUILDDIR/tmp/deploy/images/raspberry
pi3/core-image-minimal-raspberrypi3.rootfs-20260929120029.tar.bz2 -C /nfs
[sudo] Mot de passe de dev :
```

Figure 6.8 – Déploiement du rootfs dans /nfs

6.5 Retrouver l'adresse de la cible et lancer `ctris`

Comme l'adresse de la cible est attribuée par DHCP et peut changer, on la retrouve avant chaque connexion. On a d'abord vérifié que l'interface de partage de connexion était active, puis interrogé la table de voisinage :

```
nmcli device
ip neighbor show
```

L'interface `enx782d7e15da02` est bien connectée à la connexion `PARTAGE-RPi`. La table de voisinage montre une entrée `REACHABLE` sur cette interface : `10.42.0.159`, dont l'adresse MAC commence par `b8:27:eb` (préfixe des cartes Raspberry Pi). C'est donc l'adresse de la cible. On s'y connecte en SSH :

```
ssh root@10.42.0.159
```

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ nmcli device
DEVICE          TYPE      STATE                  CONNECTION
enx782d7e15da02 ethernet  connecté               PARTAGE-RPi
enp0s8          ethernet  connecté               Wired connection 1
lo              loopback  connecté (en externe)  lo
dev@debian13:~/Yocto-RPi/builds/rpi3$ ip neighbor show
10.0.3.2 dev enp0s8 lladdr 52:55:0a:00:03:02 STALE
10.42.0.159 dev enx782d7e15da02 lladdr b8:27:eb:b7:9d:7d REACHABLE
fe80::2 dev enp0s8 lladdr 52:56:00:00:00:02 router STALE
fd17:625c:f037:3::2 dev enp0s8 lladdr 52:56:00:00:00:02 router STALE
dev@debian13:~/Yocto-RPi/builds/rpi3$ ssh root@10.42.0.159

WARNING: Poky is a reference Yocto Project distribution that should be used for
testing and development purposes only. It is recommended that you create your
own distribution for production use.

root@raspberrypi3:~# ls
HELLO
root@raspberrypi3:~# ctris
root@raspberrypi3:~#
```

Figure 6.9 – Connexion SSH à la cible

La connexion aboutit à un shell root sur `raspberrypi3`, on lance ensuite le jeu :

```
ctris
```

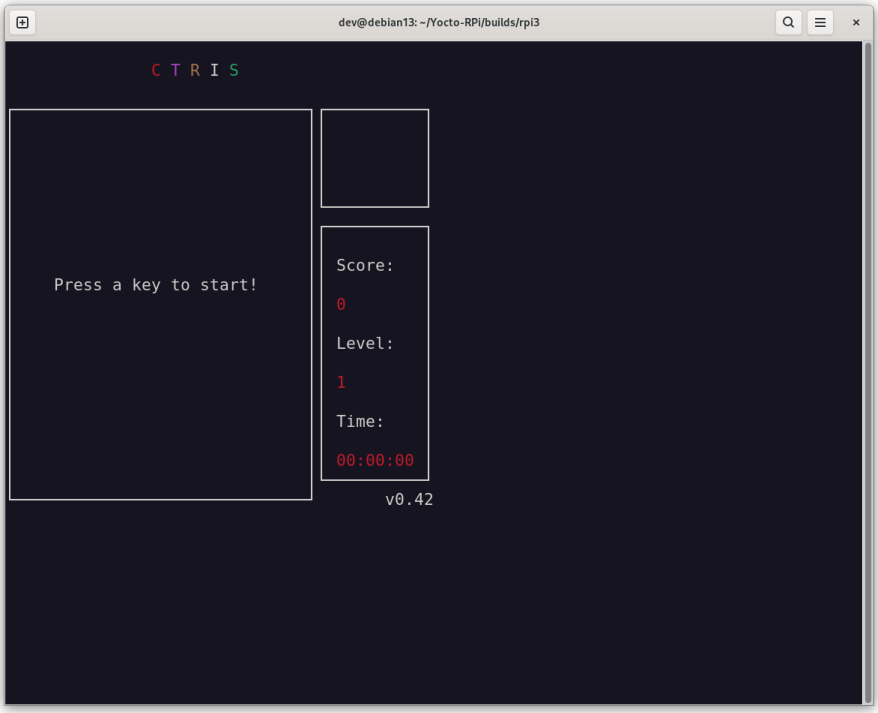


Figure 6.10 – ctris en cours d'exécution sur la cible

L'écran d'accueil de `ctris` (version 0.42) s'affiche dans le terminal, avec la zone de jeu, la pièce suivante et le tableau de score. Le jeu fonctionne donc sur la cible.

CHAPITRE 7

Lab 5 – I^2C et init

Ce lab prépare l'image pour utiliser le bus I2C de la Raspberry Pi 3, puis exploite un vrai capteur environnemental connecté sur ce bus, jusqu'à automatiser sa déclaration au démarrage via un script `init.d`.

7.1 Configuration de `local.conf`

On a complété `local.conf` avec les lignes suivantes (les autres réglages, `MACHINE`, `CMDLINE`, etc., sont ceux des labs précédents) :

```
CORE_IMAGE_EXTRA_INSTALL:append = " kernel-modules"
ENABLE_I2C = "1"
IMAGE_INSTALL:append = " i2c-tools"
```

Figure 7.1 – Ajout des configurations de I^2C

- `i2c-tools` ajoute à l'image les utilitaires nécessaires pour communiquer avec les périphériques I^2C.
- `ENABLE_I2C = "1"` est une variable de `meta-raspberrypi` qui active le bus I2C dans la configuration de démarrage de la carte.
- `CORE_IMAGE_EXTRA_INSTALL:append = " kernel-modules"` installe dans le *rootfs* l'ensemble des modules du noyau.

On a vérifié que les réglages étaient bien tous présents (et une seule fois) avec un `grep` sur `conf/local.conf` :

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ grep -nE "IMAGE\_INSTALL|EXTRA\_INSTALL|ENABLE\_I2C|CMDLINE" conf/local.conf
291:CMDLINE:append = " ip=::::yoctorpi0:eth0:dhcp "
292:CMDLINE:append = " root=/dev/nfs nfsroot=10.42.0.1:/nfs,nfsvers=3,tcp rw"
293:CMDLINE:remove = "root=/dev/mmcblk0p2 rootfstype=ext4"
295:IMAGE_INSTALL:append = " dropbear"
296:IMAGE_INSTALL:append = " ninvaders"
298:IMAGE_INSTALL:append = " ctris"
299:CORE_IMAGE_EXTRA_INSTALL:append = " kernel-modules"
300:ENABLE_I2C = "1"
301:IMAGE_INSTALL:append = " i2c-tools"
302:IMAGE_INSTALL:append = " lighttpd lighttpd-module-cgi"
```

Figure 7.2 – Vérification des réglages

7.2 Redéploiement du *rootfs* et vérification

Après reconstruction de l'image, on a vidé le répertoire exporté avant d'y ré-extraire le *rootfs*, afin de partir d'un contenu propre et de ne pas conserver de fichiers d'une ancienne version de l'image :

```
sudo rm -rf /nfs/*
sudo tar xpf $BUILDDIR/tmp/deploy/images/raspberrypi3/core-image-minimal-
    ↪ raspberrypi3.rootfs.tar.bz2 -C /nfs
```

```
find /nfs/usr/sbin -name "i2c*"
ls /nfs/lib/modules/
```

Le `find` retrouve bien les i2c-tools. Les modules du noyau 6.6.63 sont aussi bien installés.

7.3 Modules du noyau non compressés

On a écrit un petit fragment de configuration du noyau, `uncompressed_modules.cfg`, rangé dans un dossier `recipes-kernel/linux/linux-raspberrypi/` :

```
CONFIG_MODULE_COMPRESS_NONE=y
```

Un fragment `.cfg` contient quelques options du noyau que Yocto fusionne à la configuration de base lorsqu'il est listé dans le `SRC_URI` de la recette du noyau, typiquement à l'aide d'un `.bbappend`. Ici, l'option demande que les modules soient installés sous forme de fichiers `.ko` simples plutôt que compressés, il s'agit d'une nécessité, car le `modprobe` de Busybox ne sait pas gérer les modules compressés (`.ko.xz`).

7.4 Rattachement du fragment via un `.bbappend`

Ce fragment ne sert à rien tant qu'il n'est pas rattaché à la recette du noyau. On l'attache via un `.bbappend` nommé pour matcher toutes les versions de `linux-raspberrypi` par % :

```
cd ~/Yocto-RPi/meta-bootlinlabs/recipes-kernel/linux/linux-raspberrypi
nano linux-raspberrypi_%.bbappend
bitbake-layers show-appends linux-raspberrypi
```

```
dev@debian13:~/Yocto-RPi/meta-bootlinlabs/recipes-kernel/linux$ nano linux-raspberrypi_%
.bbappend
dev@debian13:~/Yocto-RPi/meta-bootlinlabs/recipes-kernel/linux$ bitbake-layers show-appe
nds linux-raspberrypi
NOTE: Starting bitbake server...
WARNING: Host distribution "debian-13" has not been validated with this version of the b
uild system; you may possibly experience unexpected failures. It is recommended that you
 use a tested distribution.
Loading cache: 100% |                                                | ETA:  --:--:--
Loaded 0 entries from dependency cache.
Parsing recipes: 100% |################################################| Time: 0:00:18
Parsing of 964 .bb files complete (0 cached, 964 parsed). 1926 targets, 80 skipped, 0 ma
sked, 0 errors.

Summary: There was 1 WARNING message.
=== Matched appended recipes ===
linux-raspberrypi_6.12.bb:
  /home/dev/Yocto-RPi/meta-bootlinlabs/recipes-kernel/linux/linux-raspberrypi_%.bbappend
linux-raspberrypi_6.1.bb:
  /home/dev/Yocto-RPi/meta-bootlinlabs/recipes-kernel/linux/linux-raspberrypi_%.bbappend
linux-raspberrypi_6.6.bb:
  /home/dev/Yocto-RPi/meta-bootlinlabs/recipes-kernel/linux/linux-raspberrypi_%.bbappend
dev@debian13:~/Yocto-RPi/meta-bootlinlabs/recipes-kernel/linux$
```

Figure 7.3 – Rattachement du .bbappend au noyau

Les trois recettes (`linux-raspberrypi_6.1.bb`, `6.6.bb` et `6.12.bb`) apparaissent bien comme *matched*, chacune pointant vers notre `.bbappend`.

7.5 Reconstruction de l'image

```
bitbake core-image-minimal
```

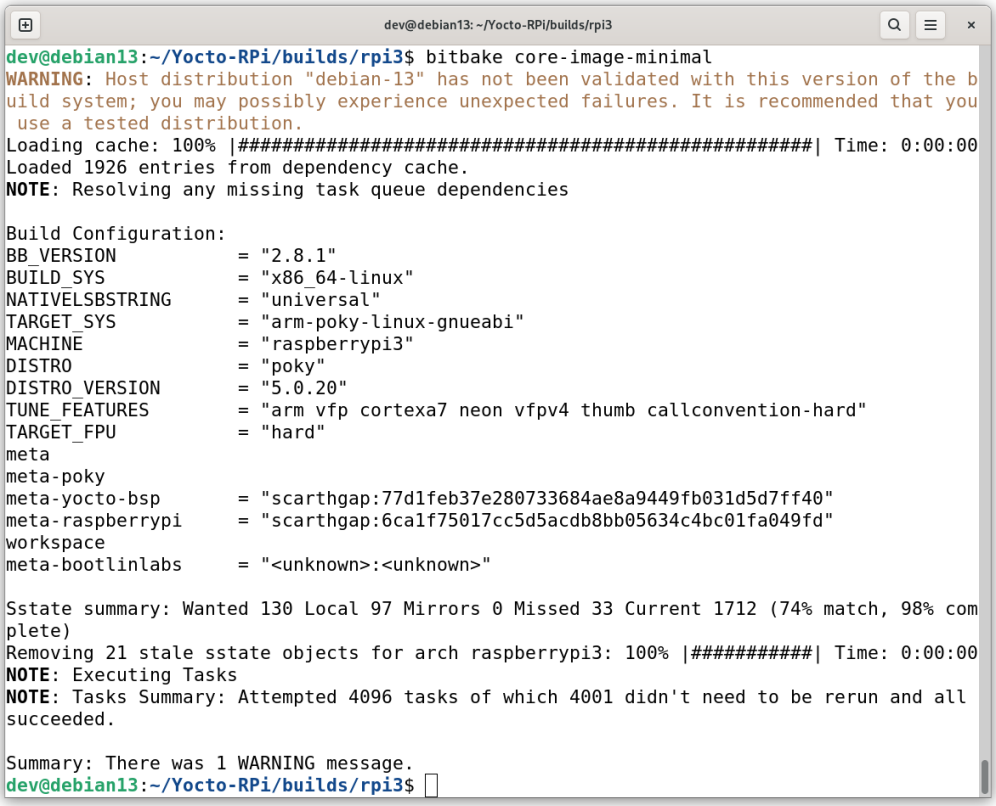


Figure 7.4 – Reconstruction de l'image avec le fragment noyau

7.6 Activation du bus I2C côté *firmware*

Après avoir réécrit la carte SD, on a vérifié que `ENABLE_I2C = "1"` avait bien été traduit dans le `config.txt` du bootloader, sur la partition de boot :

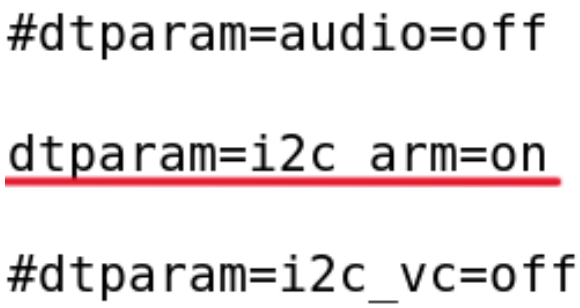


Figure 7.5 – config.txt sur la partition de boot

La ligne `dtparam=i2c_arm=on` y figure bien, décommentée, c'est elle qui active concrètement le contrôleur I2C matériel au démarrage du firmware, avant même que Linux ne démarre.

7.7 Détection du capteur sur le bus

Après redémarrage, on charge le pilote générique I2C et on scanne le bus 1 :

```
modprobe i2c-dev
i2cdetect -y -r 1
```

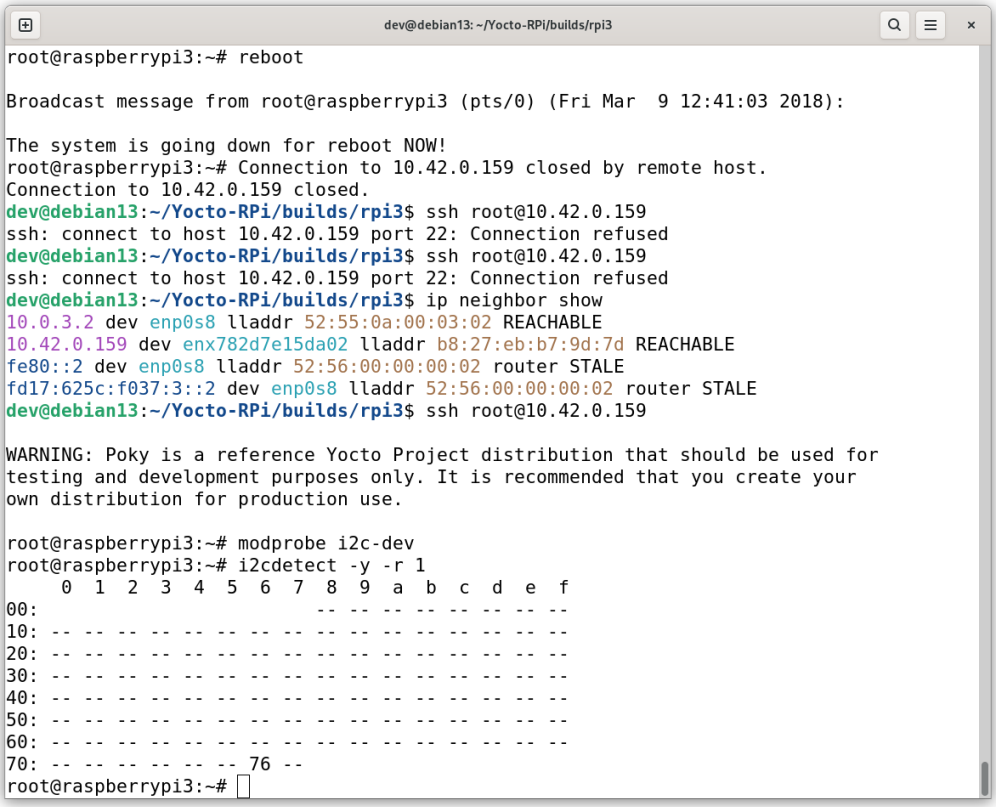


Figure 7.6 – Détection du capteur avec i2cdetect

Un périphérique répond à l'adresse `0x76`. On tente d'abord de le déclarer comme un `bmp680` à cette adresse :

```
echo "bmp680 0x76" > /sys/bus/i2c/devices/i2c-1/new_device
```

```
root@raspberrypi3:~# i2cdetect -y -r 1
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f
00:                         -- -- -- -- -- -- -- --
10: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
20: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
30: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
40: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
50: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
60: -- -- -- -- -- -- -- -- -- -- -- -- -- -- -- --
70: -- -- -- -- -- -- 76 --
root@raspberrypi3:~# echo "bme680 0x76" > /sys/bus/i2c/devices/i2c-1/new_device
root@raspberrypi3:~# lsmod | grep bme680
bme680_i2c             12288  0
bme680_core            16384  1 bme680_i2c
industrialio           86016  2 bmp280,bme680_core
root@raspberrypi3:~# dmesg | tail -n 10
[  905.027143] bmp280 1-0076: supply vddd not found, using dummy regulator
[  905.035057] bmp280 1-0076: supply vdda not found, using dummy regulator
[  905.046001] bmp280 1-0076: bad chip id: 0x61 is not a known chip id
[  922.349763] i2c i2c-1: Failed to register i2c client bmp680 at 0x76 (-16)
[  947.274307] i2c i2c-1: delete_device: Deleting device bmp280 at 0x76
[  949.283525] i2c i2c-1: new_device: Instantiated device bmp680 at 0x76
[  998.851029] i2c i2c-1: Failed to register i2c client bmp680 at 0x76 (-16)
[ 1035.948132] i2c i2c-1: delete_device: Deleting device bmp680 at 0x76
[ 1079.855395] i2c i2c-1: delete_device: Can't find device in list
[ 1087.502279] i2c i2c-1: new_device: Instantiated device bme680 at 0x76
root@raspberrypi3:~#
```

Figure 7.7 – Déclaration du capteur bmp680

7.8 Lecture des mesures via l'interface IIO

```
root@raspberrypi3:~# ls /sys/bus/iio/devices/iio\:device0/
in_humidityrelative_input               in_temp_oversampling_ratio
in_humidityrelative_oversampling_ratio  name
in_pressure_input                       oversampling_ratio_available
in_pressure_oversampling_ratio          power
in_resistance_input                     subsystem
in_temp_input                           uevent
root@raspberrypi3:~#
```

Figure 7.8 – Fichiers IIO du capteur dans sysfs

Le sous-système IIO (*Industrial I/O*) expose directement les grandeurs mesurées sous forme de fichiers texte, sans qu'il soit nécessaire d'écrire la moindre ligne de code applicatif. On y lit en particulier la température, la pression, l'humidité relative et la résistance du capteur de gaz :

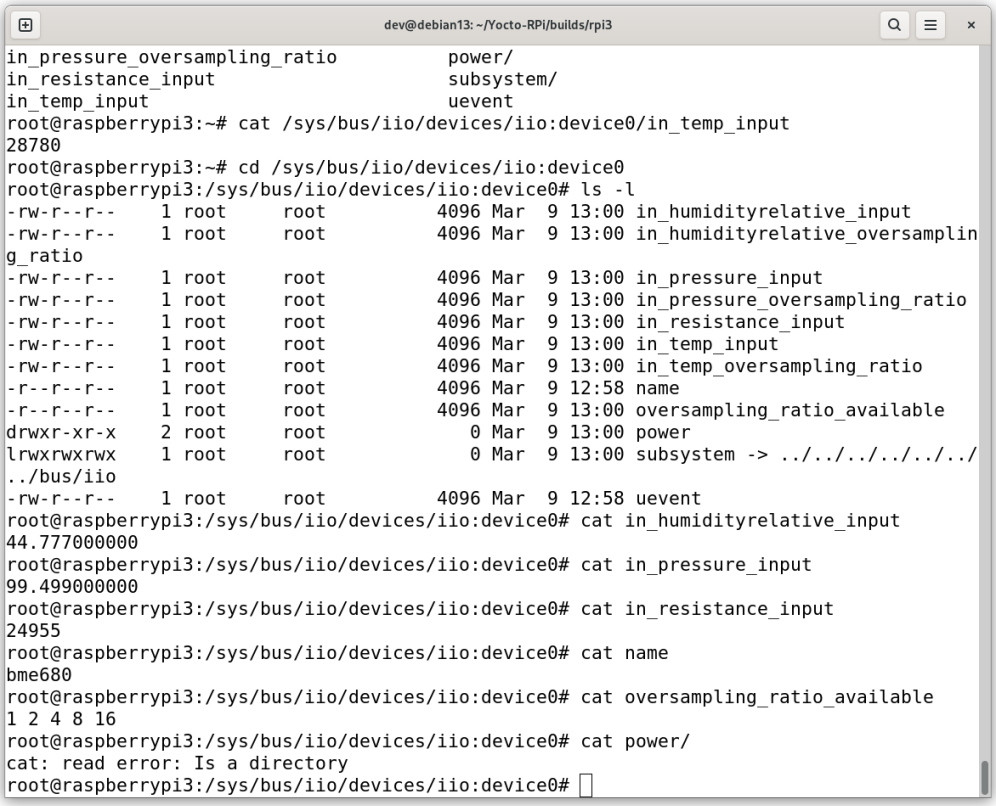


Figure 7.9 – Lecture des mesures du capteur

7.9 Automatisation avec un script `init.d`

Déclarer le capteur à la main à chaque démarrage n'est pas viable. On installe un script `/etc/init.d/bme680`, suivant la convention LSB (*Provides*, *Default-Start/Stop*), qui déclare le capteur au démarrage et le retire proprement à l'arrêt :

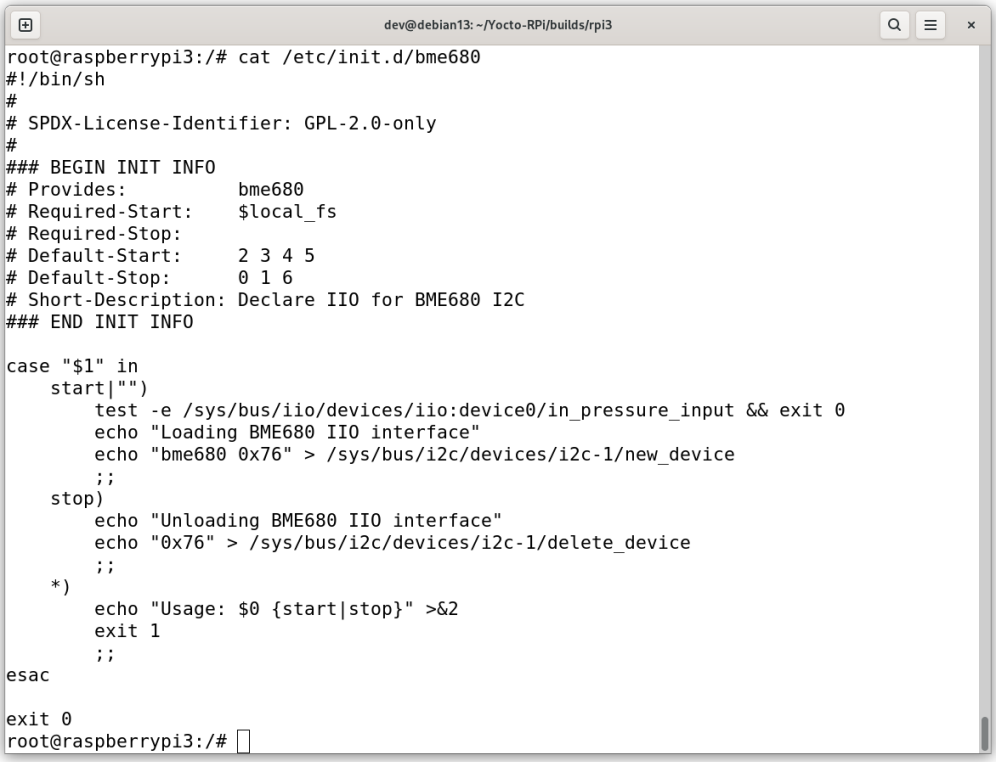


Figure 7.10 – Script init.d bme680

Le script teste d'abord si l'interface IIO existe déjà (`test -e .../in_pressure_input`) avant de la recréer.

On le rend exécutable et on l'enregistre dans les niveaux d'exécution avec `update-rc.d` :

```
chmod +x /etc/init.d/bme680
update-rc.d bme680 defaults 99
```

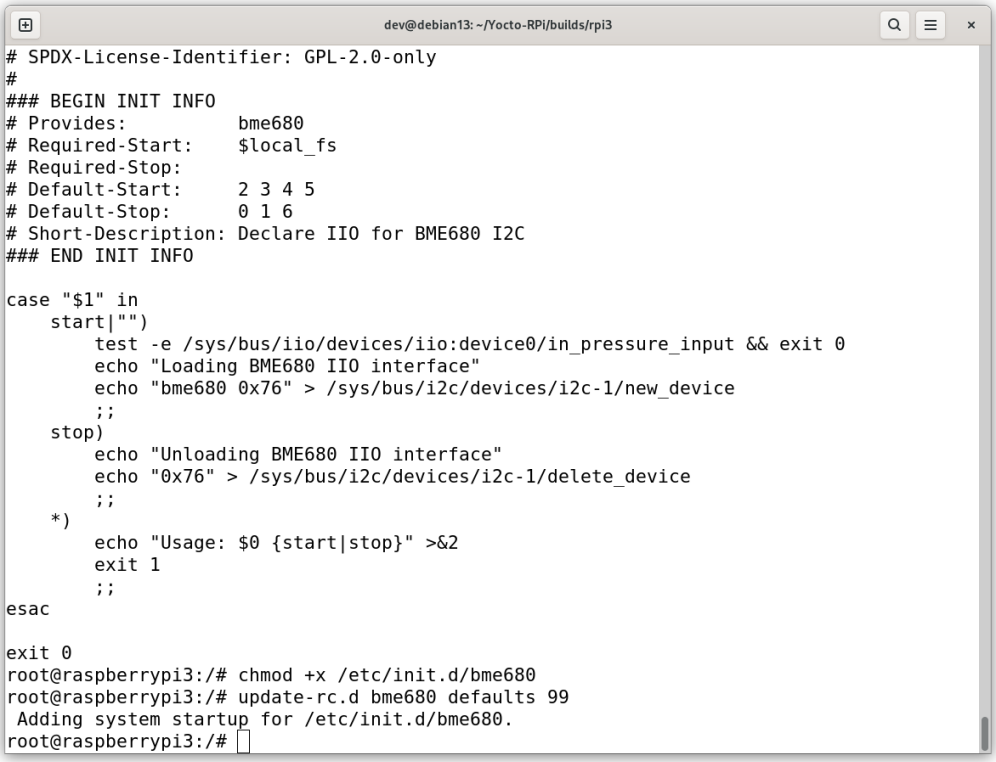


Figure 7.11 – Enregistrement du script avec update-rc.d

7.10 Vérification après redémarrage

```
reboot
```

```
dev@debian13:~/Yocto-RPi/builds/rpi3$ ssh root@10.42.0.159

WARNING: Poky is a reference Yocto Project distribution that should be used for
testing and development purposes only. It is recommended that you create your
own distribution for production use.

root@raspberrypi3:~# cat /sys/bus/iio/devices/iio:device0/in_temp_input
28570
root@raspberrypi3:~#
```

Figure 7.12 – Lecture de la température après redémarrage

Après le redémarrage, une simple lecture de `in_temp_input` renvoie immédiatement une valeur cohérente (28570, soit 28.57°C), sans qu'on ait eu besoin de redéclarer le capteur à la main : le script d'init s'en est chargé automatiquement.

CHAPITRE 8

Lab Web – Interface web de télémétrie

Ce dernier lab expose les mesures du capteur BME680 lues via l'interface IIO sous la forme d'un petit tableau de bord web, consultable depuis n'importe quel navigateur du réseau local.

8.1 Architecture globale

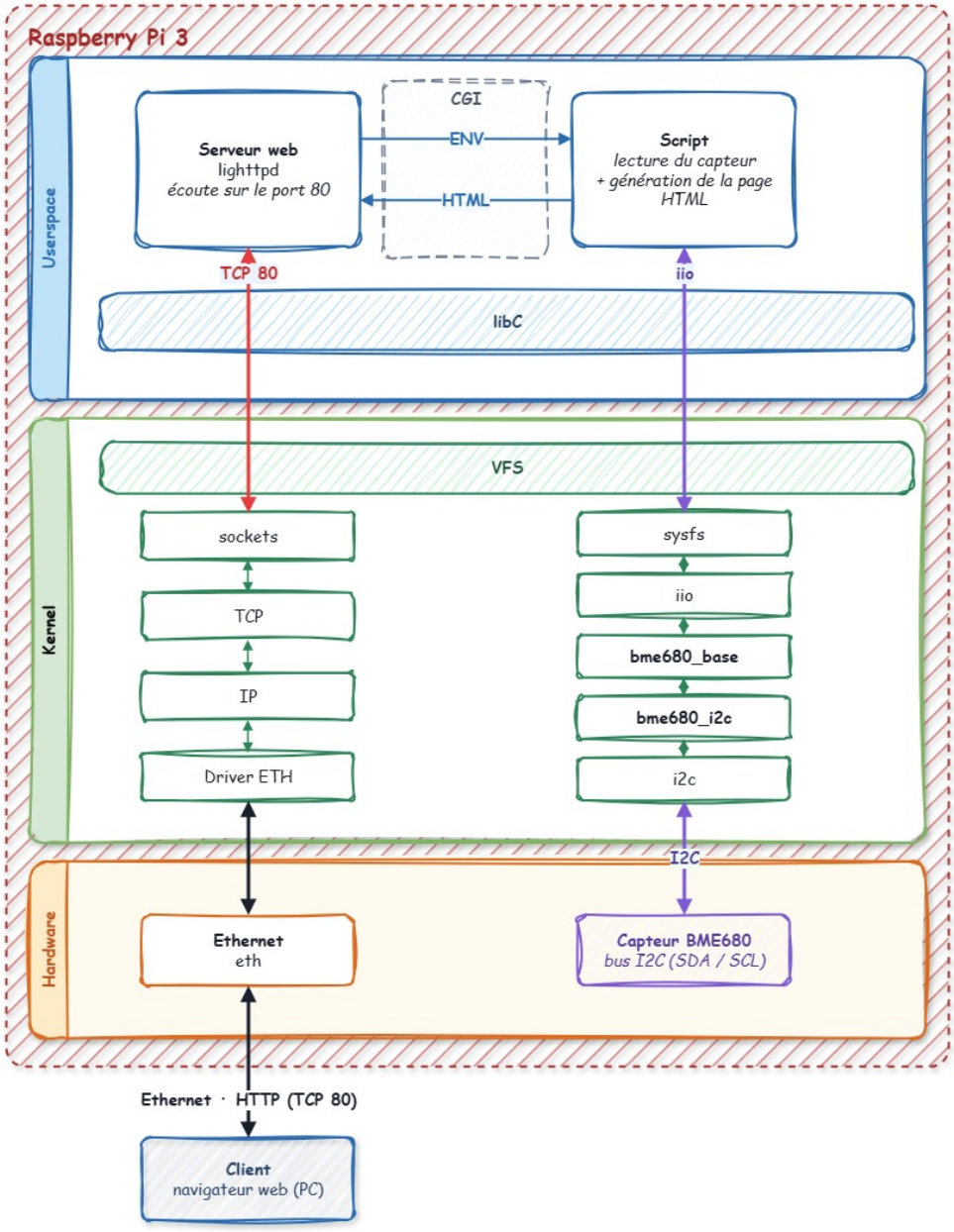


Figure 8.1 – Architecture globale de l'interface web

Flux d'une requête web :

1. Le navigateur envoie une requête HTTP sur le port TCP 80 (Ethernet → pile réseau →

socket).

2. lighttpd exécute le script via CGI (la requête est transmise par l'environnement : ENV).
3. Le script lit les mesures dans sysfs, via libC et le VFS (interface IIO).
4. Le driver IIO (bme680_base + bme680_i2c) interroge le BME680 sur le bus I2C.
5. Le script renvoie du HTML à lighttpd, qui le renvoie au navigateur.

8.2 Choix et configuration du serveur web

On utilise `lighttpd`, un serveur HTTP léger bien adapté à l'embarqué, avec son module CGI pour pouvoir exécuter un script shell à chaque requête plutôt que de servir uniquement du contenu statique :

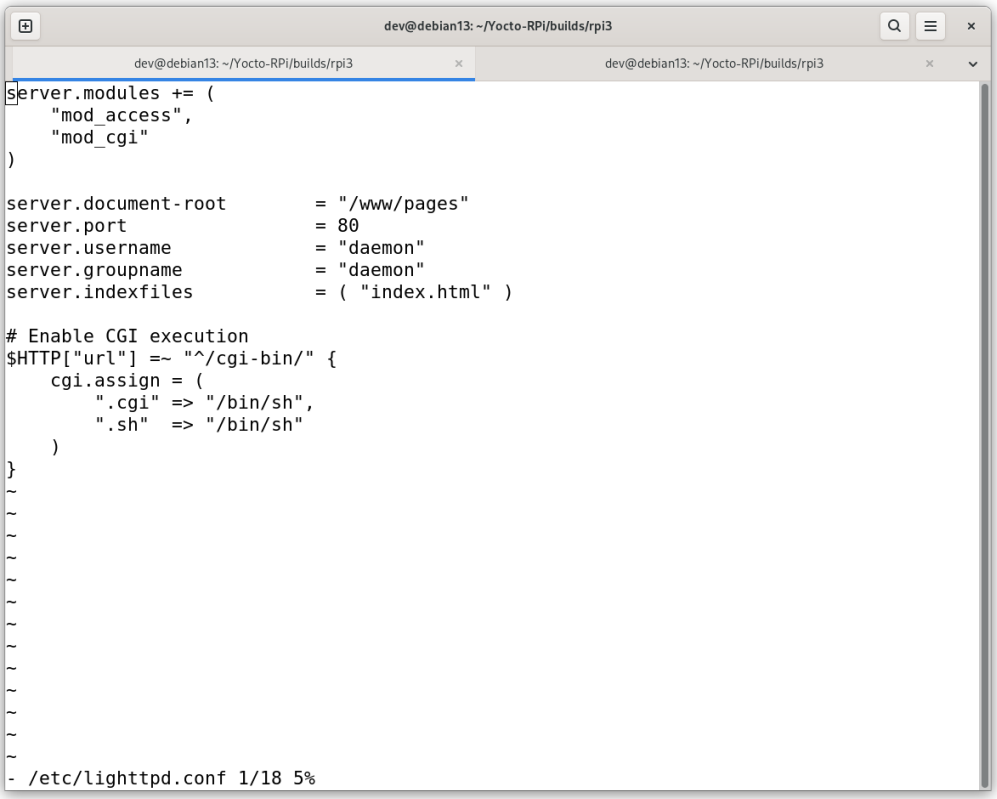


Figure 8.2 – Configuration de lighttpd

La racine documentaire est fixée à `/www/pages`, et tout ce qui est demandé sous `/cgi-bin/` avec une extension `.cgi` ou `.sh` est exécuté par `/bin/sh` plutôt que renvoyé tel quel : c'est ce mécanisme CGI qui permet de générer une réponse dynamique (les valeurs du capteur) à chaque appel, au lieu d'une page figée.

8.3 Script CGI de lecture du capteur

Le script `/www/pages/cgi-bin/data.cgi` fait le lien entre le sous-système IIO du noyau (vu au Lab 5) et le monde web : il lit les fichiers texte exposés par le pilote, les convertit dans les unités attendues, et renvoie le résultat au format JSON :

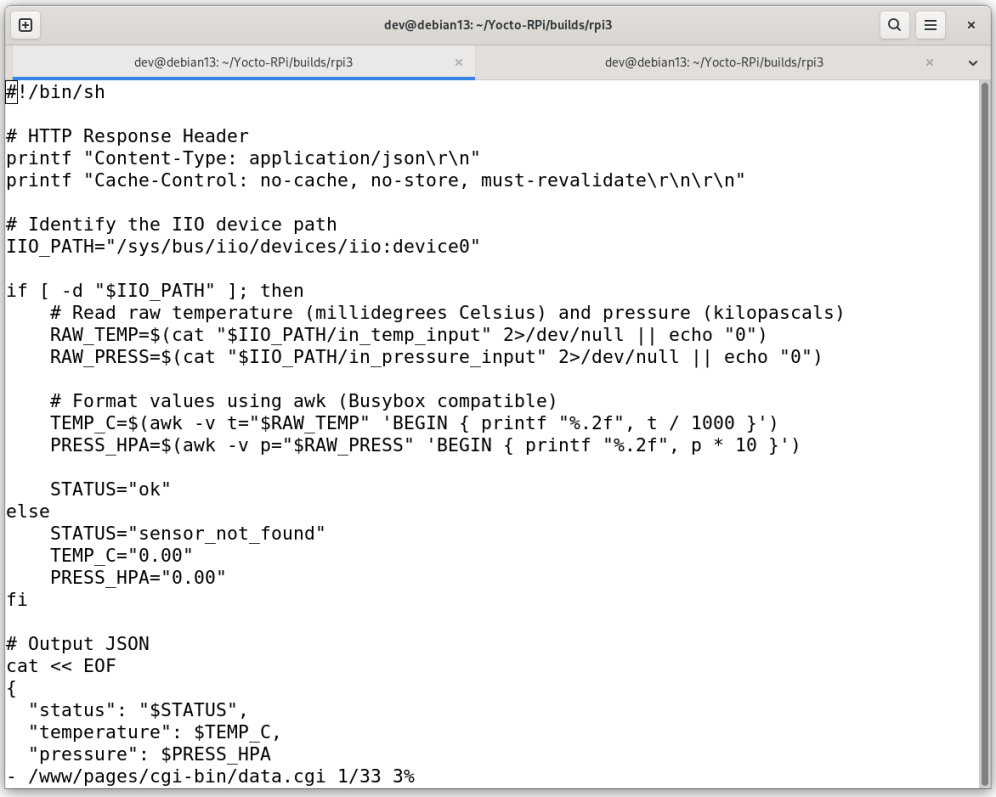


Figure 8.3 – Script CGI data.cgi

8.4 Page web statique

La page `/www/pages/index.html` fournit l'habillage visuel : un thème sombre, une carte centrée affichant deux indicateurs (température et pression), et un peu de JavaScript (non montré ici) qui interroge périodiquement `/cgi-bin/data.cgi` pour rafraîchir les valeurs sans recharger la page.

dev@debian13: ~/Yocto-RPi/builds/rpi3

dev@debian13: ~/Yocto-RPi/builds/rpi3 dev@debian13: ~/Yocto-RPi/builds/rpi3

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Raspberry Pi 3 - Sensor Monitor</title>
    <style>
        body {
            font-family: sans-serif;
            background-color: #0f172a;
            color: #f8fafc;
            display: flex;
            flex-direction: column;
            align-items: center;
            justify-content: center;
            min-height: 100vh;
            margin: 0;
        }
        .container {
            background-color: #1e293b;
            padding: 2rem;
            border-radius: 12px;
            width: 320px;
            text-align: center;
        }
        .metric-card {
            background-color: #334155;
            padding: 1.2rem;
            margin-bottom: 1rem;
            border-radius: 8px;
- /www/pages/index.html 1/91 1%
```

Figure 8.4 – Page web index.html

8.5 Résultat final

En se connectant depuis un navigateur du réseau local à l'adresse de la carte, on obtient le tableau de bord attendu :

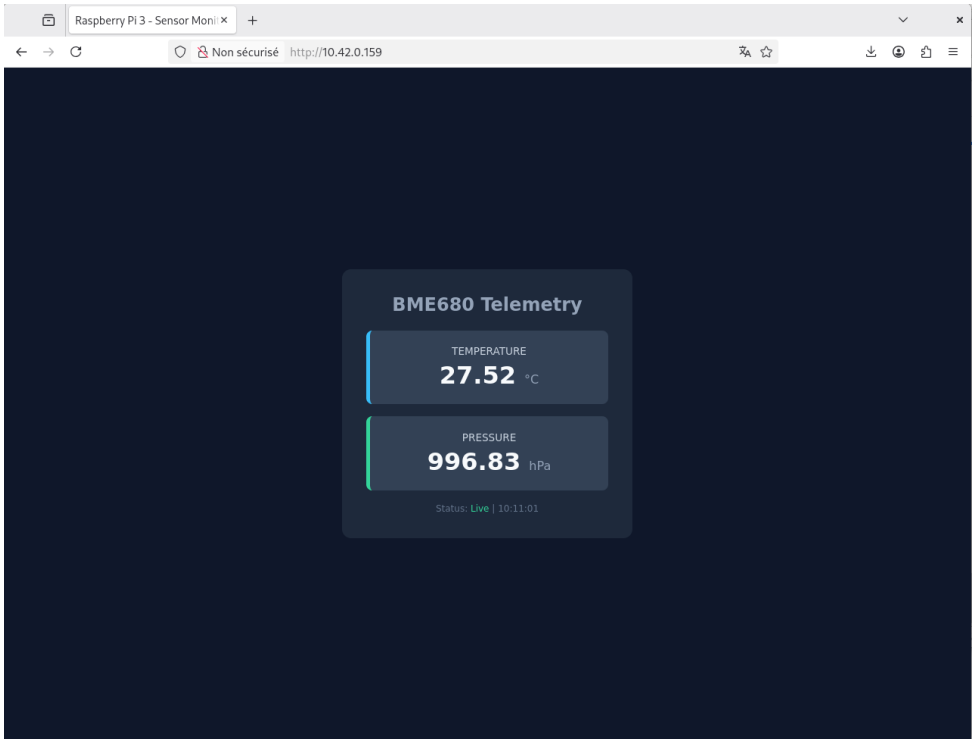


Figure 8.5 – Tableau de bord BME680 Telemetry

La page *BME680 Telemetry* affiche la température (27.52°C) et la pression (996.83 hPa) mesurées en temps réel par le capteur, avec un indicateur de statut *Live* et l'heure de la dernière mise à jour. Ceci confirme que toute la chaîne fonctionne de bout en bout : capteur I2C → pilote noyau IIO → script CGI → page web, entièrement embarquée sur la Raspberry Pi elle-même.

Conclusion

La réalisation de ces Labs nous a permis de développer une bonne compréhension du **Yocto Project** et de son utilisation pour construire une distribution Linux embarquée adaptée à une cible matérielle comme la Raspberry Pi 3.

À travers la construction d'images, la configuration du réseau et du démarrage par NFS, l'ajout d'applications personnalisées, la création d'une couche `meta-bootlinlabs` et l'utilisation de `devtool`, nous avons acquis une expérience pratique des principaux mécanismes de Yocto/OpenEmbedded. Nous avons également pu mettre en œuvre la communication avec un capteur BME680 via le bus I^2C et exploiter ses mesures depuis Linux jusqu'à une interface web de télémétrie. :contentReference

Certaines étapes ont nécessité plusieurs essais, notamment lors de la compilation et de l'intégration de certaines recettes. La résolution de ces problèmes nous a permis de mieux comprendre le fonctionnement de BitBake, la gestion des dépendances et l'organisation des différentes couches. Au-delà des commandes utilisées, ce TP nous a ainsi permis de mieux comprendre la chaîne complète de développement, du système de build jusqu'à l'exploitation des périphériques sur une plateforme embarquée.